

Airport Management System

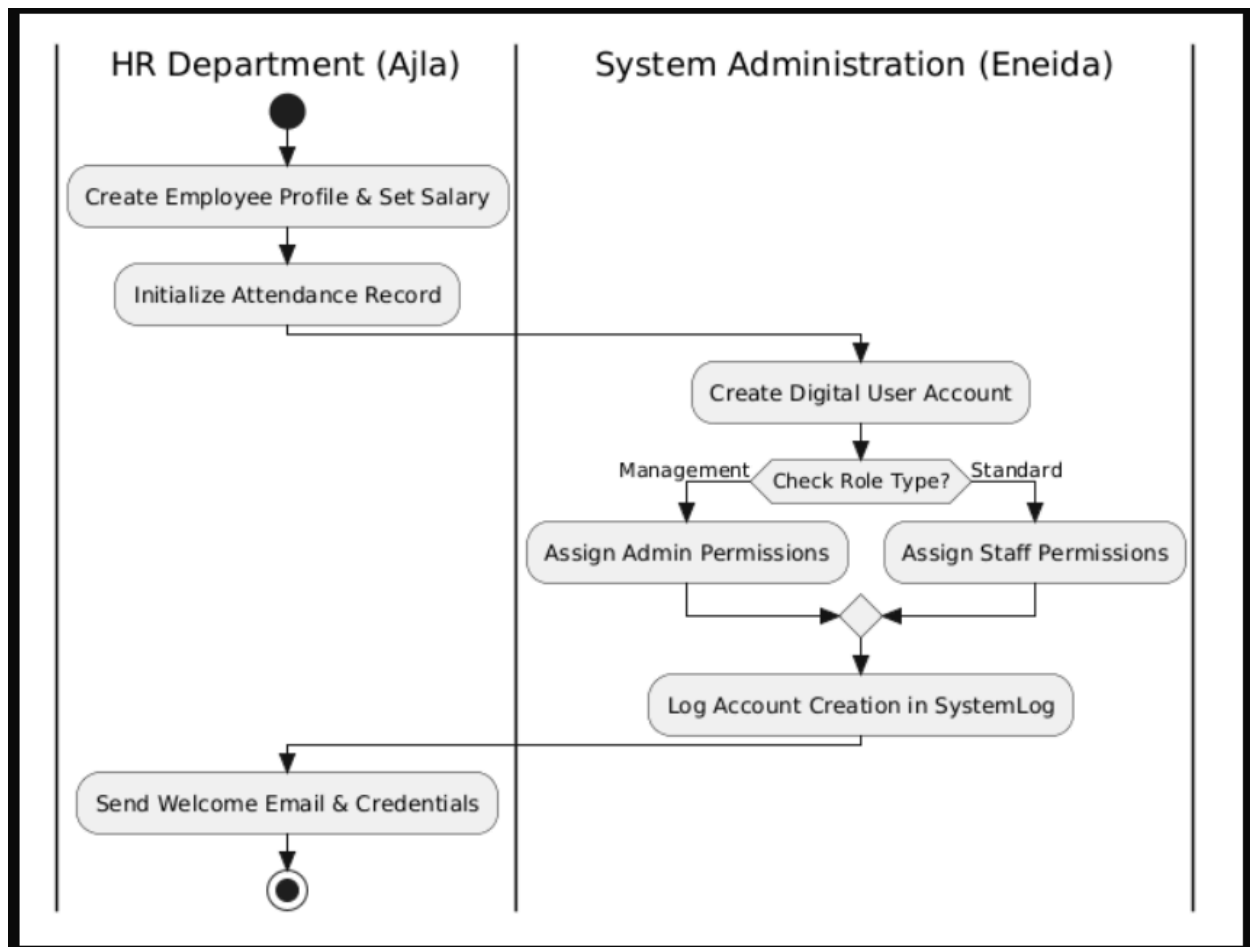
Business Process Modeling (BPMN) Specifications

Prepared by: Ajla Alcani & Eneida Balliu

Course: Software Modeling and Design Date: April 16, 2026

Document Objective

The purpose of this section is to outline the high-level business workflows of the Airport Management System. While our previous UML models focused on software objects and database interactions, these Business Process Model and Notation (BPMN) diagrams map the cross-functional operational procedures and physical handoffs between distinct airport departments. Please note that the following diagram represents the first of three main business processes modeled for this assignment.



Airport Management System

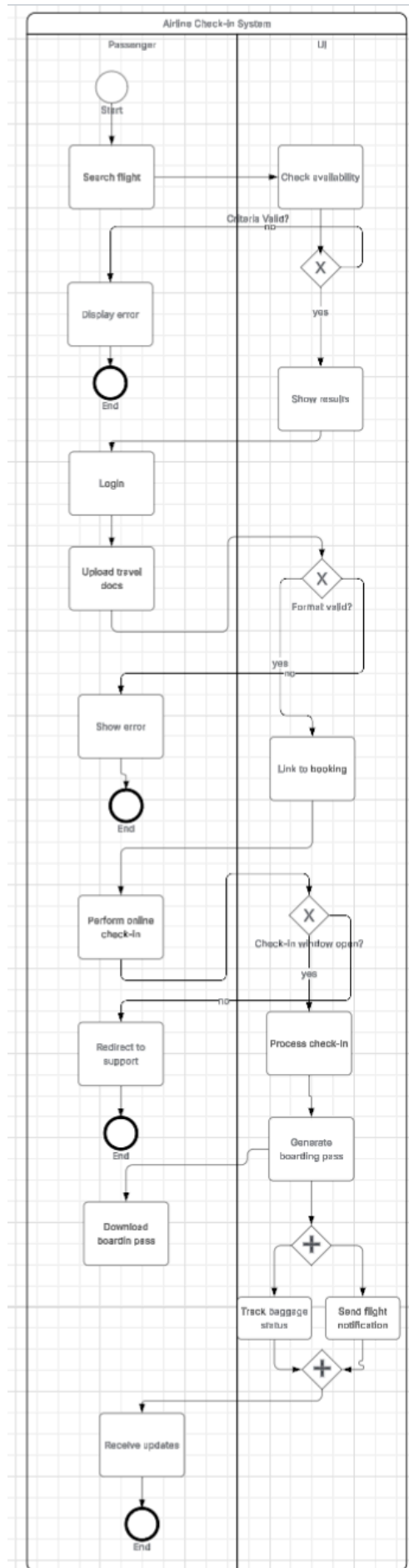
Business Process Modeling (BPMN) Specifications

Prepared by: Betül Kaan

Course: Software Modeling and Design Date: April 16, 2026

Document Objective

The purpose of this section is to outline the high-level business workflows of the Airport Management System. While our previous UML models focused on software objects and database interactions, these Business Process Model and Notation (BPMN) diagrams map the cross-functional operational procedures and physical handoffs between distinct airport departments.



Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Ajla Alcani

Document Objective

*The primary objective of this document is to detail the structural organization and message flow of the **Human Resources and Customer Support Modules** through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the passage of time, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between objects.*

Technical Scope

This document focuses on the interaction network between:

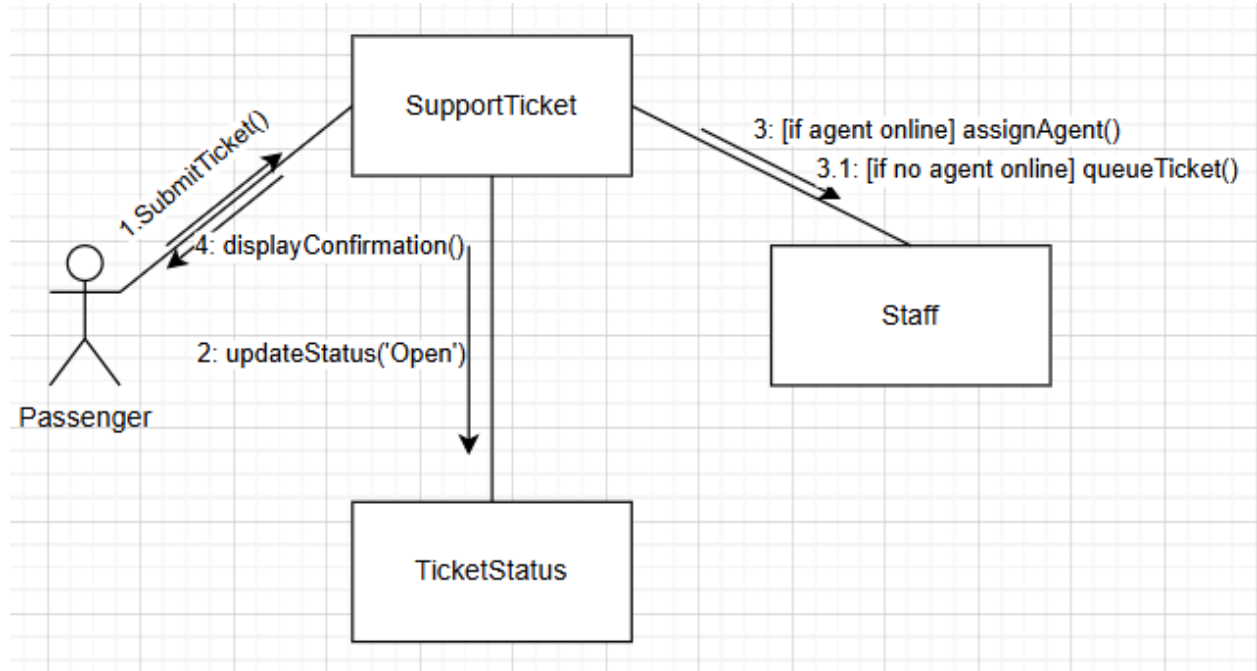
- 1. **Actors:** HR Admin, Employee (Staff), and Passenger.*
- 2. **Control/Boundary Objects:** System elements responsible for routing requests (e.g., ChatSession, SupportTicket, Payroll).*
- 3. **Entity Objects:** The database layer including TicketStatus, Staff, and AttendanceRecord.*

System Interaction Standards

To maintain technical consistency, every collaboration diagram in this document adheres to the following architectural rules:

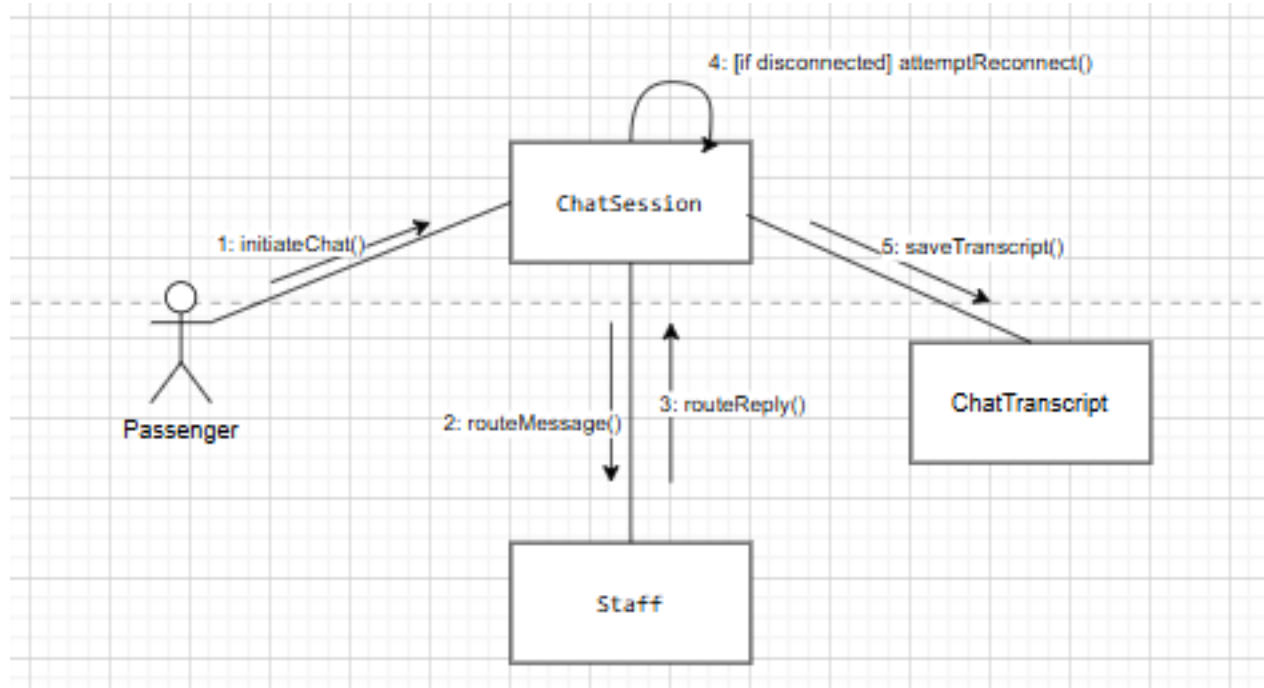
- 1. **Strict UML Notation:** All models utilize standardized UML collaboration/communication symbols.*
- 2. **Structural Links:** Solid lines represent the structural pathways over which objects can communicate.*
- 3. **Hierarchical Numbering:** Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2, 2.1) to clearly indicate nested operations and sequential logic.*
- 4. **Controller-Centric Routing:** Central objects are used to route messages between actors and data entities, keeping the architecture decoupled and highly cohesive.*

1. Submit Support Ticket



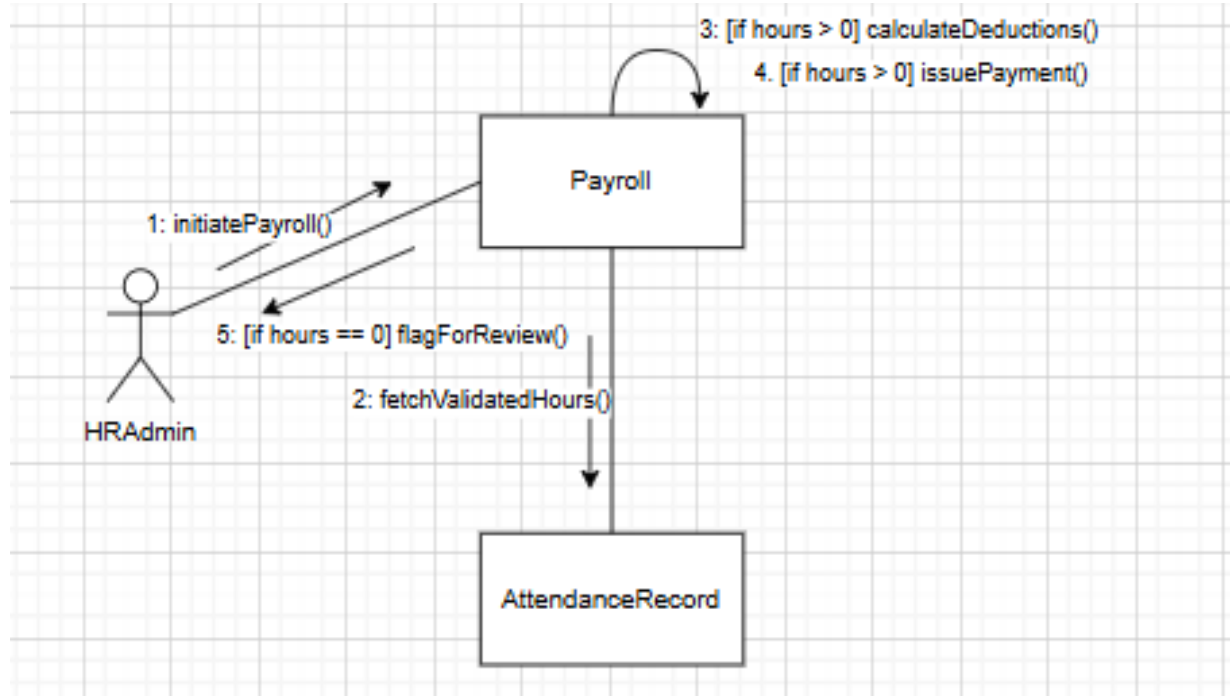
- **Focus:** Structural Message Routing for Ticket Resolution
- **Description:** This collaboration diagram maps the direct structural links between the **Passenger** actor, the core **SupportTicket** object, the assigned **Staff** agent, and the **TicketStatus** history entity.
- **Key Logic:** The numbered messages dictate the strict execution sequence. The **SupportTicket** acts as the central control object. Upon receiving the initial creation trigger (1), it sequentially logs its open state (1.1). It then routes the assignment to an available agent (2). Once the agent resolves the issue (3), the ticket ensures data compliance by logging its final resolved state (3.1).

2. Live Chat Session



- **Focus:** Synchronous Message Routing and Data Preservation
- **Description:** This collaboration diagram illustrates the structural communication pathways required to facilitate real-time chat between a Passenger and a Customer Support Agent (*Staff*), using the *ChatSession* as the central mediating object.
- **Key Logic:** The decimal numbering explicitly tracks the synchronous back-and-forth of the conversation. The passenger initiates the connection (1), which immediately prompts the session to notify an agent (1.1). During the active conversation loop, passenger messages (2) are structurally routed to the agent (2.1), and agent replies are routed back (2.2). Finally, when the passenger ends the session (3), the *ChatSession* object executes a self-message to save the transcript (3.1), fulfilling data retention requirements without needing external triggers.

3. Process Payroll

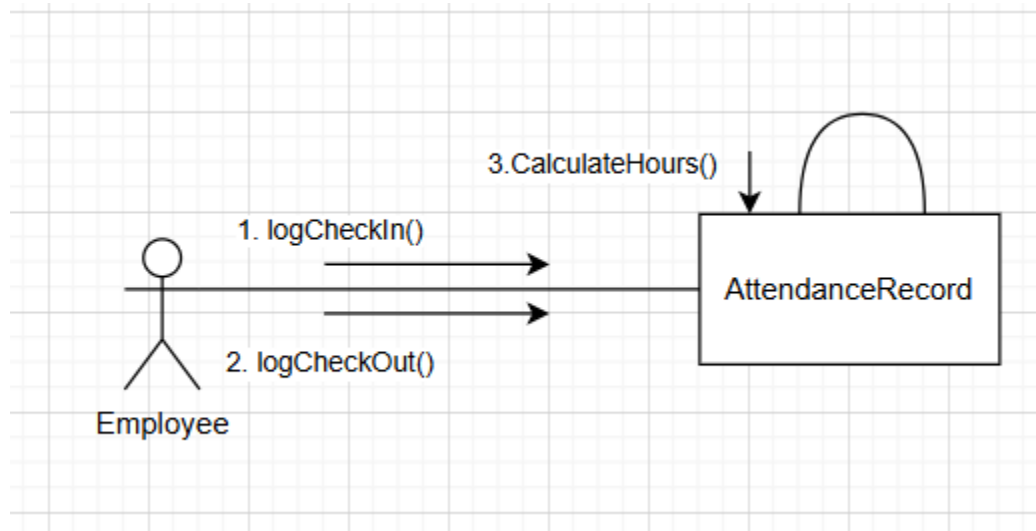


Focus: Internal Data Retrieval and Financial Processing

Description: This collaboration diagram maps the structural relationships required for salary disbursement. It highlights how the core **Payroll** object acts as an autonomous controller, sitting structurally between the HR Admin actor and the raw employee data.

Key Logic: The execution sequence guarantees strict data integrity. When the HR Admin initiates the process (1), the **Payroll** object does not immediately issue payment. Instead, it is structurally forced to route a request to the **AttendanceRecord** to calculate and return the validated hours (1.1). Only after receiving this external data does the **Payroll** object execute its internal self-messages to calculate deductions (2) and safely issue the final payment (3).

4. Log Staff Attendance



Focus: Structural Time Tracking and Automated Processing

Description: This collaboration diagram depicts the direct structural interaction between the Employee actor and the *AttendanceRecord* entity for tracking shift durations.

Key Logic: The interaction is divided into two distinct primary events. The employee initiates the sequence by routing a check-in message to the system (1). At the end of the shift, the employee structurally routes a check-out message (2). This second message instantly triggers a nested, internal system operation within the *AttendanceRecord* to calculate the total hours worked (2.1), ensuring automated and mathematically accurate timekeeping without manual HR intervention.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Eneida Balliu

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the System Administration Module through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the temporal order of events, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between system objects to ensure a robust and scalable management framework.

Technical Scope

This document focuses on the interaction network between:

- 1. Actor: System Administrator.*
- 2. Control/Boundary Objects: System elements responsible for routing administrative requests (e.g., AccountController, AirportController, MonitoringController).*
- 3. Entity Objects: The persistence layer including UserAccount, AirportResource, Flight, and the SystemLog.*

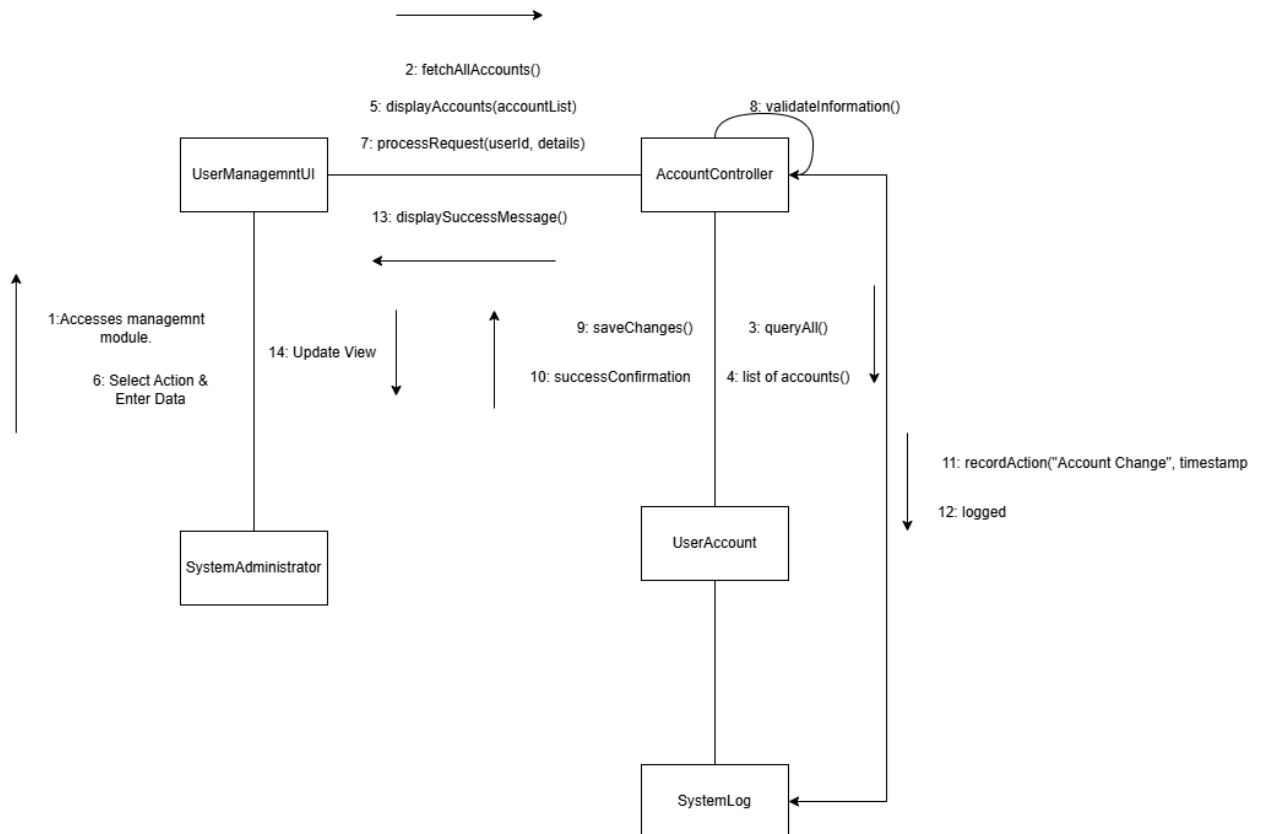
System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this document adheres to the following architectural rules:

- 1. Strict UML Notation: All models utilize standardized UML collaboration/communication symbols (Object Boxes, Link Lines, and Message Arrows).*
- 2. Structural Links: Solid lines represent the static structural pathways over which objects are permitted to communicate.*
- 3. Hierarchical Numbering: Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.*
- 4. Controller-Centric Routing: Centralized Control objects are used to decouple the User Interface (Boundary) from the Database layer (Entities), ensuring high cohesion and low coupling across the management system.*

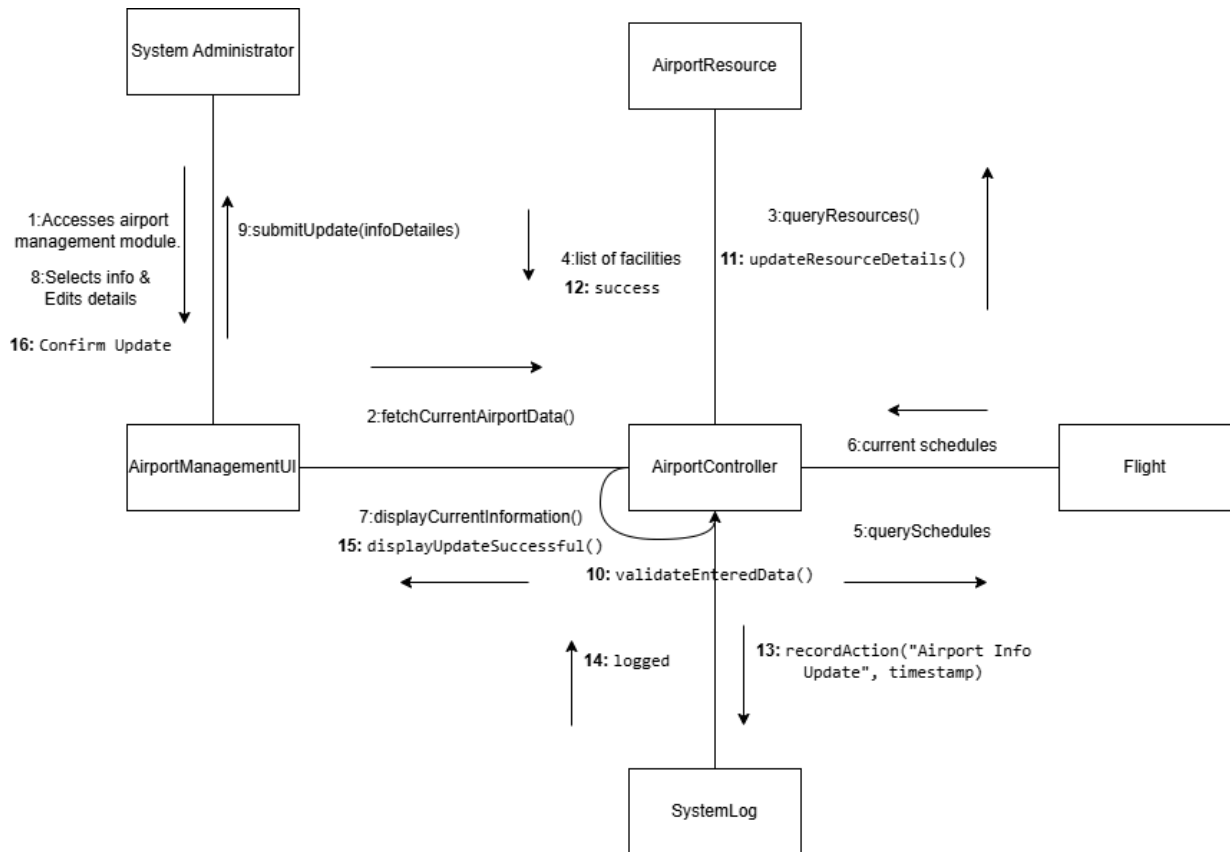
UC_40: Manage User Accounts

- *Focus: CRUD operations for administrative security.*
- *Description: Illustrates the lifecycle of a user account from the UI request through the AccountController to database persistence in the UserAccount entity.*
- *Key Logic: Centralized validation (7) ensures data integrity before any saveChanges() operation is permitted.*



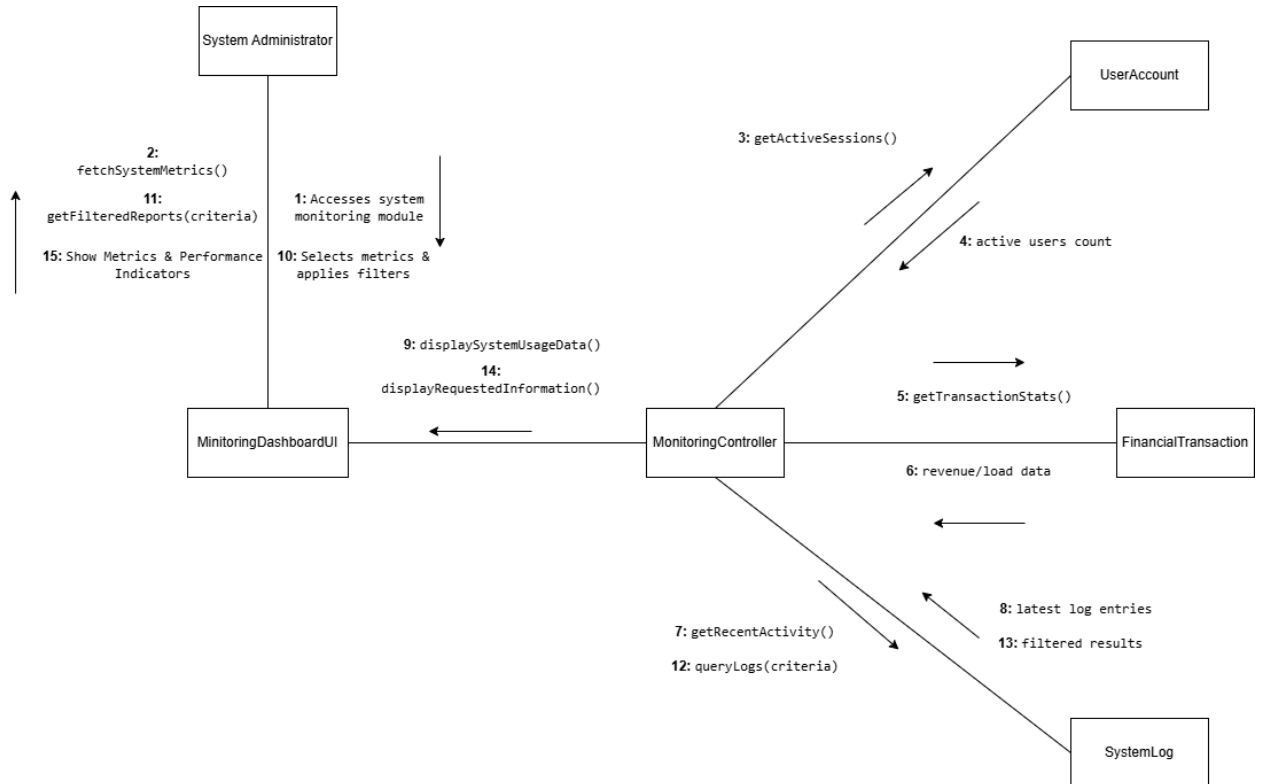
UC_41: Update Airport Information

- *Focus: Multi-entity synchronization of airport infrastructure.*
- *Description: Models the AirportController as a coordinator that pulls and pushes data across both AirportResource (facilities) and Flight (schedules) objects.*
- *Key Logic: Divergent message routing (3 & 4) allows a single administrative action to update disparate system components simultaneously.*



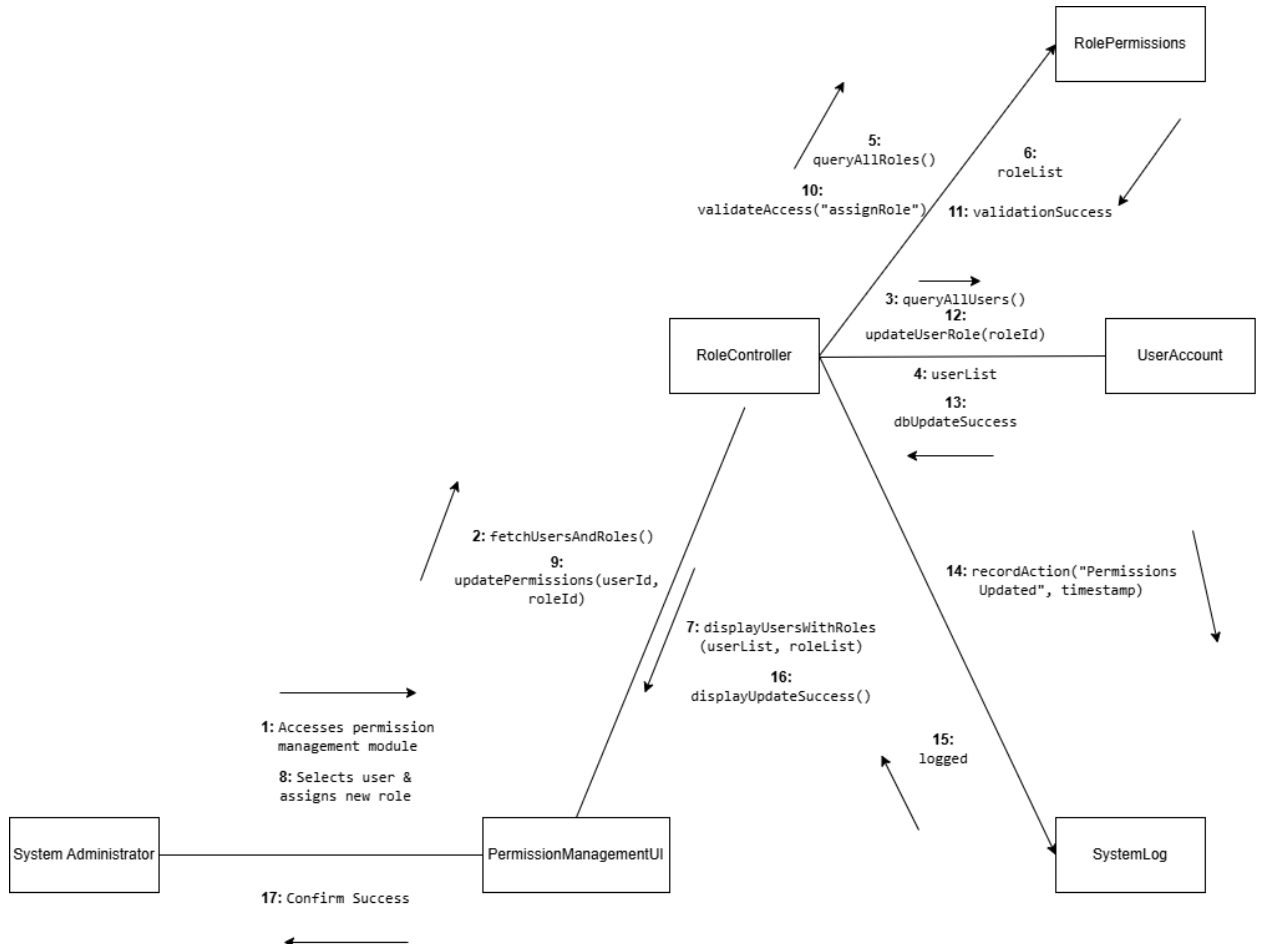
UC_42: Monitor System Usage

- *Focus: Real-time data aggregation and performance visibility.*
- *Description: Demonstrates a high-fan-out retrieval pattern where the MonitoringController queries UserAccount, FinancialTransaction, and SystemLog to populate a unified dashboard.*
- *Key Logic: Read-only data flow (3, 4, 5) optimized for low-latency reporting without altering the state of the entity objects.*



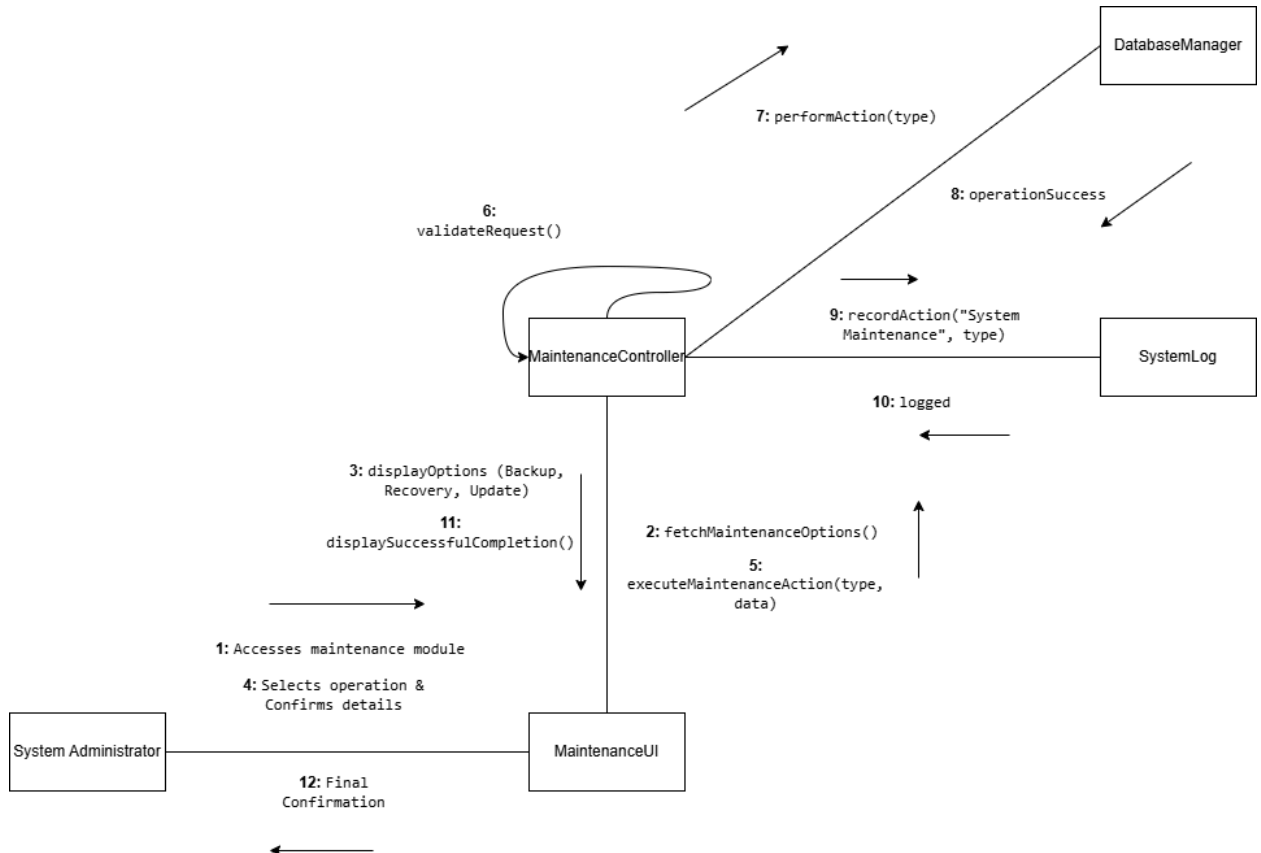
UC_43: Manage System Permissions

- *Focus: Role-Based Access Control (RBAC) enforcement.*
- *Description: Details the structural path for modifying access rights, requiring coordination between UserAccount profiles and RolePermissions definitions.*
- *Key Logic: Nested validation (10) ensures that permission changes are checked against existing security policies before the user record is updated.*



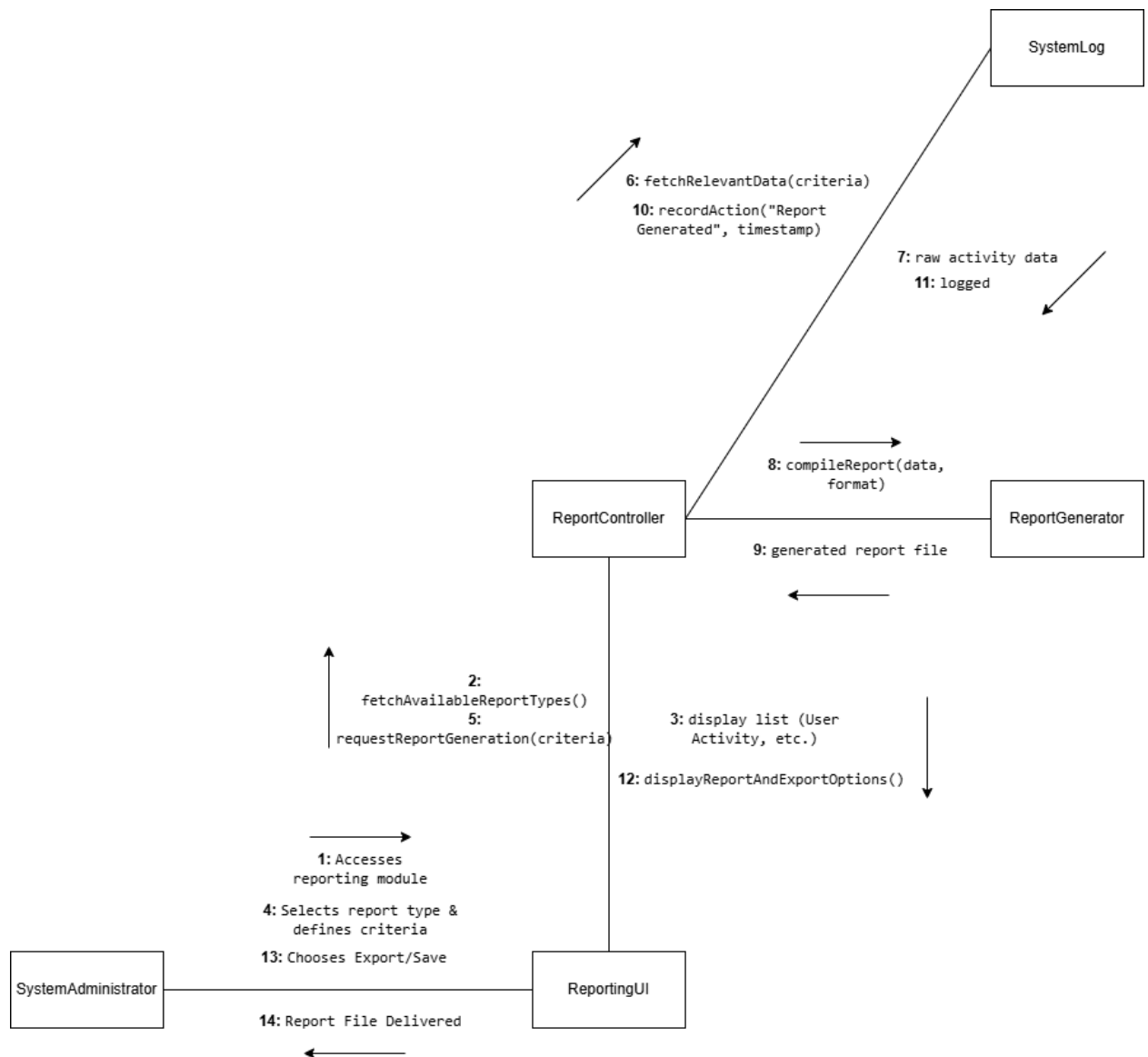
UC_44: Perform System Maintenance

- *Focus:* Database health and disaster recovery operations.
- *Description:* Traces the execution of critical maintenance tasks (backup/restore) handled by a specialized DatabaseManager utility via the MaintenanceController.
- *Key Logic:* Atomic execution flow—the system ensures a successful operationSuccess (8) return from the database before logging the activity.



UC_45: Generate System Activity Reports

- *Focus: Data compilation and professional auditing.*
- *Description: Shows the interaction between the ReportController and a dedicated ReportGenerator utility to transform raw SystemLog data into formatted documents.*
- *Key Logic: Sequential processing where raw data is first fetched (6), then passed to a utility object (8) for transformation, keeping the controller decoupled from formatting logic.*



Airport Management System

Flight Operations Management – Use Case Specifications

Course: Software Modeling and Design

Date: March 20, 2026

Prepared by: Albina Blloshmi

Document Objective

The primary objective of this document is to present a structured analysis of the Baggage Management Module within the Airport Management System through a collection of formal Use Cases.

The document models the interaction between passengers, airport staff, and the system during baggage handling operations, including baggage claiming, tracking, delayed baggage processing, baggage delivery, and baggage information updates.

These Use Cases define the operational workflow, alternative flows, validation procedures, and system requirements necessary to ensure efficient, secure, and reliable baggage management services.

Technical Scope

This document focuses on the operational interaction between:

1. Actors

- *Passenger*
- *Airport Staff*
- *System*

2. Functional Components

- *Baggage Tracking Module*
- *Claim Processing Module*
- *Delivery Management Module*
- *Notification and Monitoring Services*

3. Entity Objects

- *BaggageRecord*
 - *PassengerAccount*
 - *FlightInformation*
 - *DeliveryStatus*
 - *SystemLog*
-

System Interaction Standards

To maintain consistency and academic rigor, every Use Case in this document follows these standards:

1. Structured Use Case Format

Each Use Case contains:

- *Summary*
- *Dependency*
- *Actors*
- *Preconditions*
- *Main Sequence*
- *Alternative Sequence*
- *Non-functional Requirements*
- *Postconditions*

2. Sequential Logic

All operational steps are described in chronological order to clearly define actor-system interaction.

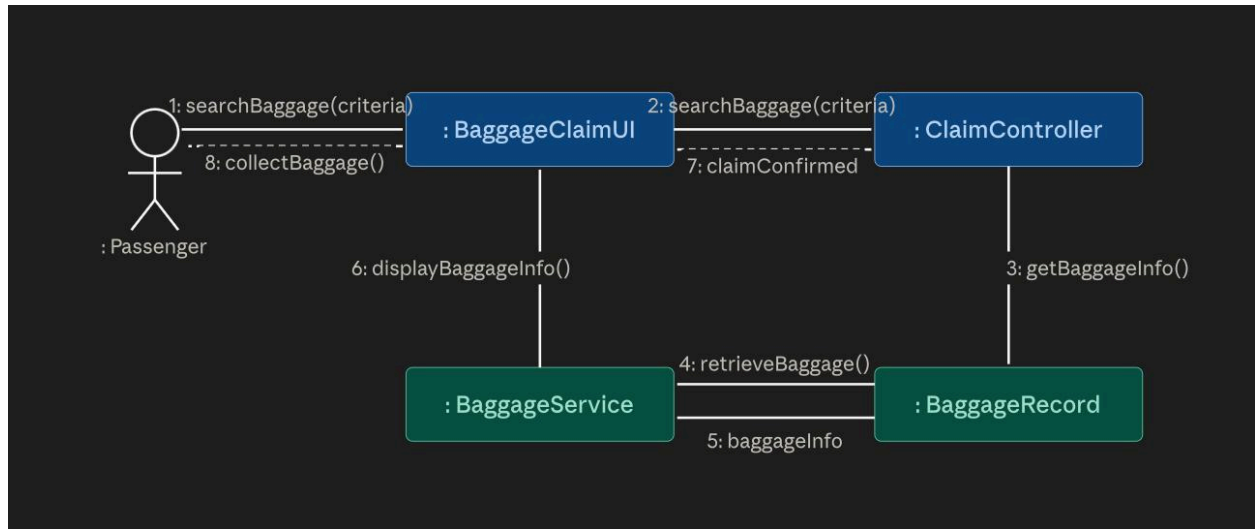
3. Validation Control

System validation ensures data accuracy before processing baggage operations, tracking requests, claims, or updates.

4. Secure System Processing

All baggage operations are executed through authenticated system procedures to maintain security and data integrity.

UC-08: Claim Baggage at Destination



Focus

Passenger baggage retrieval process after flight arrival.

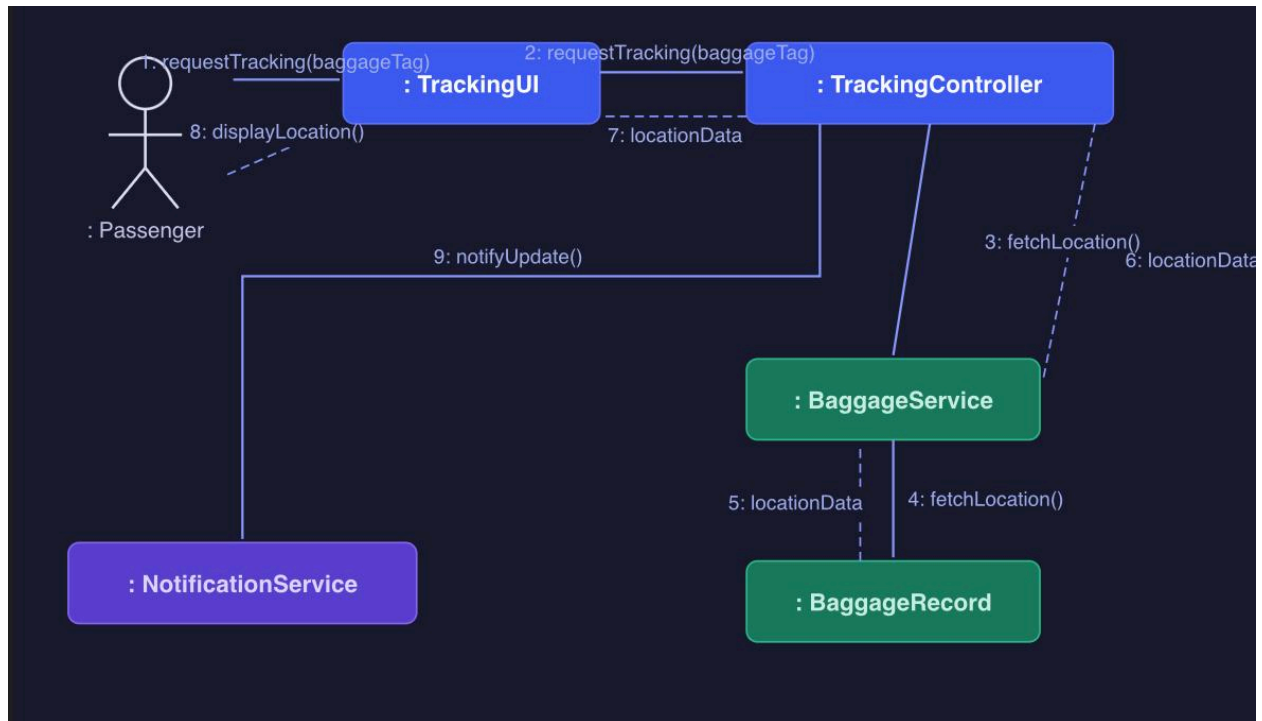
Description

This Use Case models the workflow in which a passenger retrieves baggage from the designated baggage claim area after arriving at the destination airport.

Key Logic

The system coordinates baggage carousel assignment and supports issue reporting if baggage cannot be located.

UC-09: Track Baggage in Real-Time



Focus

Real-time baggage location monitoring.

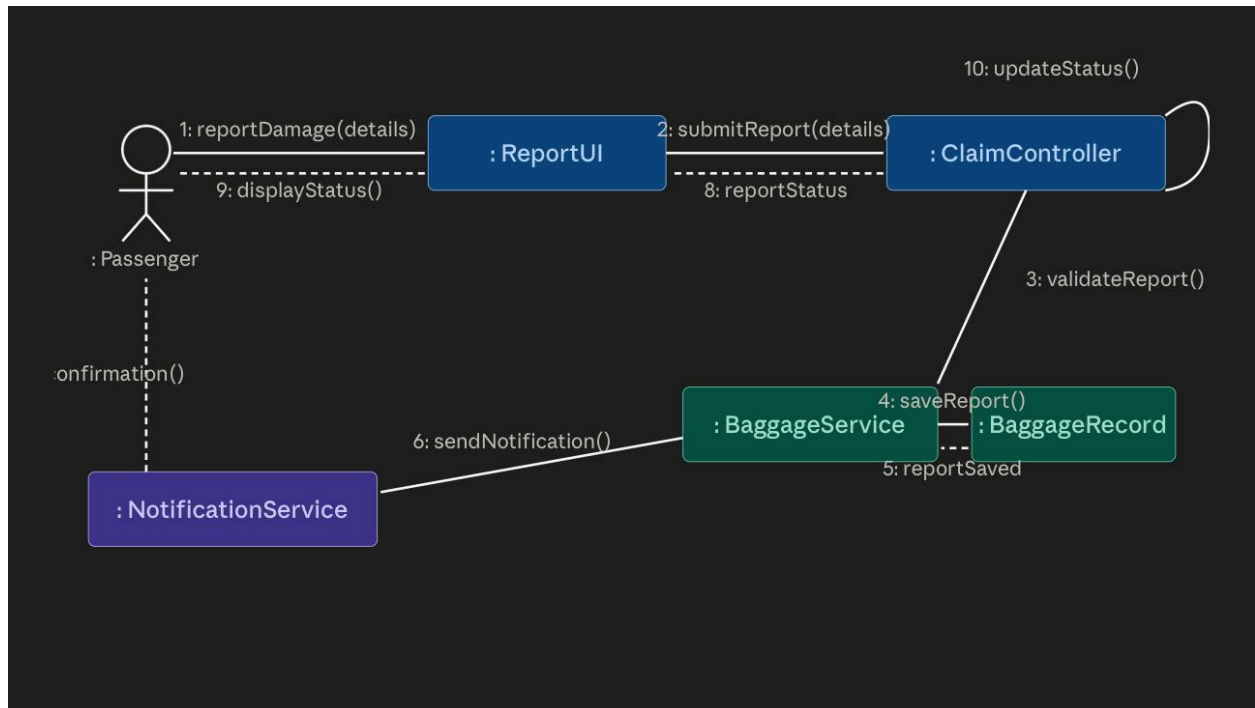
Description

This Use Case illustrates how passengers monitor baggage movement and status updates throughout the journey using the tracking system.

Key Logic

The tracking module continuously retrieves live baggage data and displays the latest known location to the passenger.

UC-10: Report Damaged Baggage



Focus

Damage reporting and claim processing.

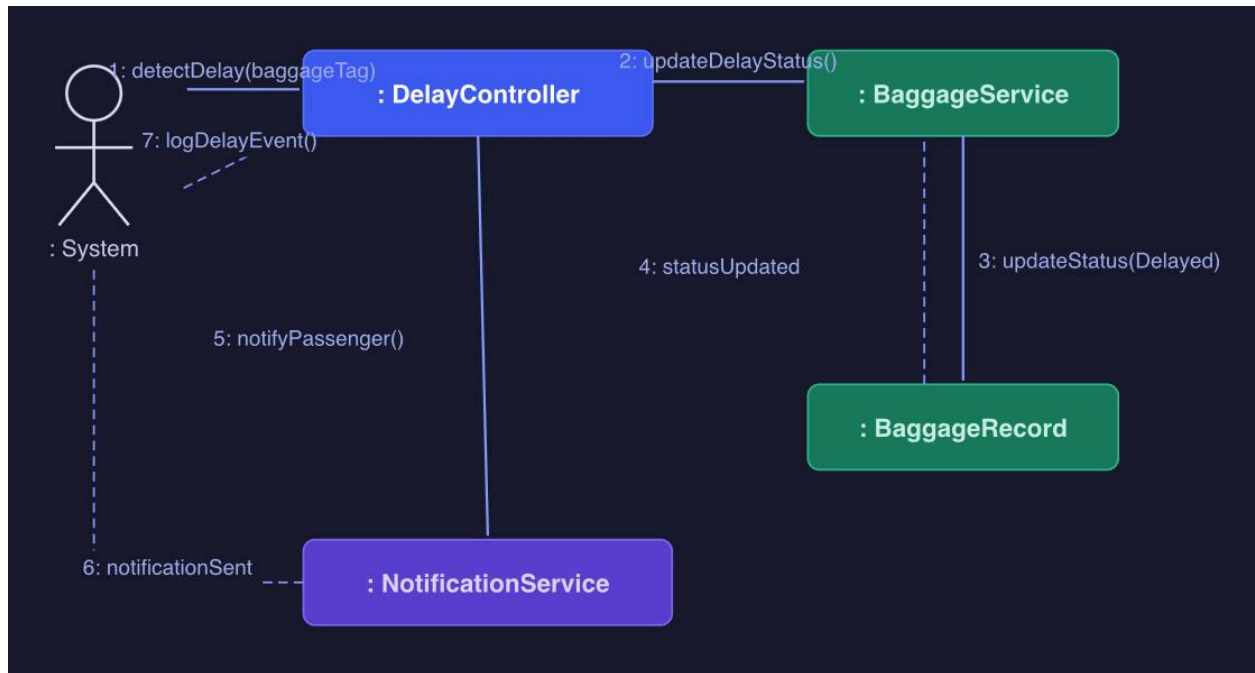
Description

This Use Case models the process where passengers report damaged baggage after retrieval and airport staff verify and process the complaint.

Key Logic

Centralized verification ensures that only valid baggage damage claims are accepted and processed.

UC-11: Handle Delayed Baggage



Focus

Management of delayed baggage situations.

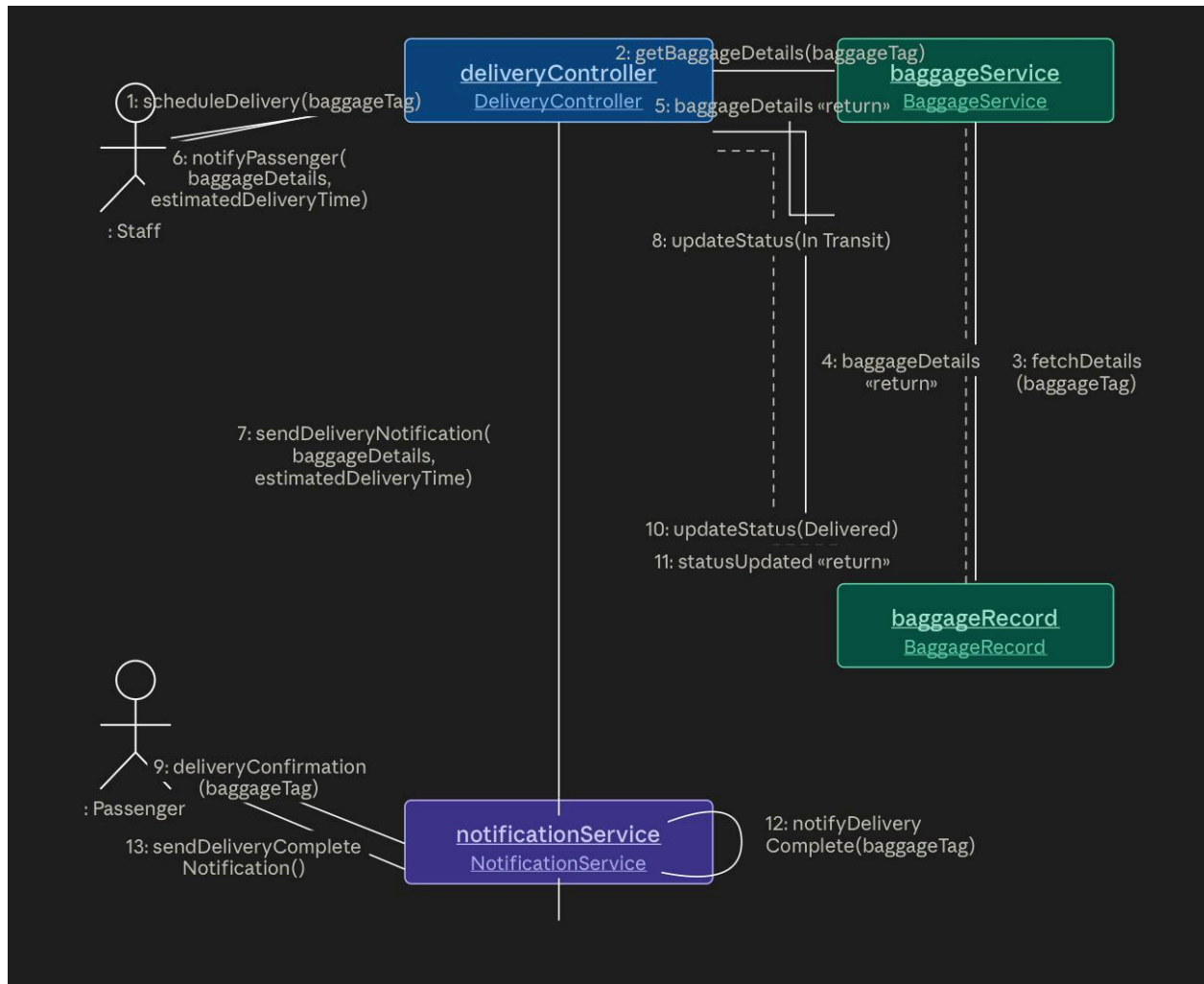
Description

This Use Case describes how the system detects baggage delays, updates baggage status, notifies passengers, and initiates recovery procedures.

Key Logic

Automated notifications and tracking updates minimize passenger uncertainty and improve operational efficiency.

UC-12: Deliver Baggage to Passenger



Focus

Baggage delivery coordination after delay recovery.

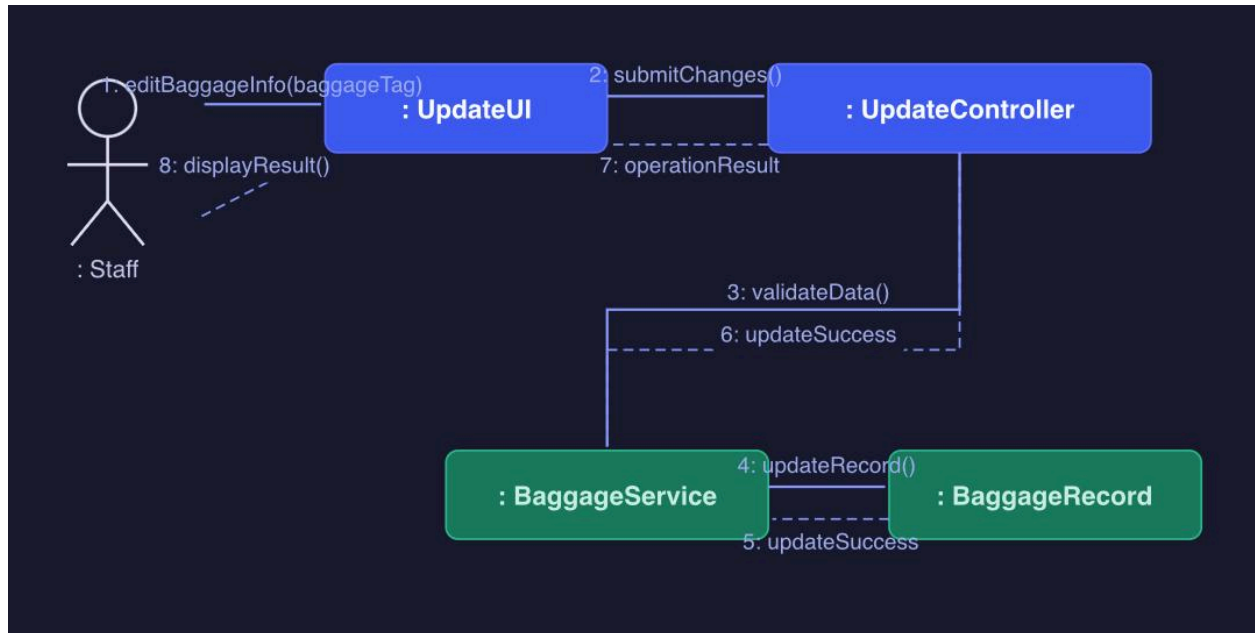
Description

This Use Case models the delivery workflow where located baggage is transported and delivered to the passenger.

Key Logic

The delivery subsystem coordinates scheduling, status updates, and delivery confirmation.

UC-13: Update Baggage Information



Focus

Modification and maintenance of baggage records.

Description

This Use Case illustrates how airport staff access and update baggage information within the system database.

Key Logic

Input validation and secure access control ensure accurate and protected baggage data management.

Conclusion

This document provides a structured representation of baggage management operations within the Airport Management System. The presented Use Cases define the interaction logic, validation flow, and operational responsibilities required to support secure, accurate, and efficient baggage handling services for both passengers and airport staff.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Andri Mucllari

Document Objective

*The primary objective of this document is to define the structural organization and message flow of the **Finance and Billing Management subsystem** within the Airport Management System. This is achieved through a set of Collaboration (Communication) Diagrams.*

Unlike Sequence Diagrams, which focus on the chronological order of interactions, these Collaboration Diagrams emphasize the architectural relationships between system components and the routing of messages across objects involved in financial operations such as payments, transaction monitoring, and billing processes.

This document focuses on the interaction network between the following components:

Actors:

Finance Staff, Airport Staff, Passenger

Control/Boundary Objects:

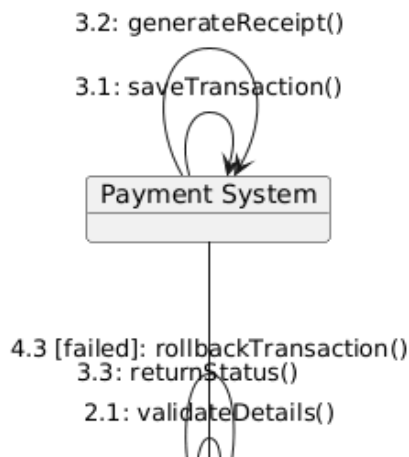
Core system components responsible for handling and routing financial operations, such as PaymentSystem, BillingController, TransactionMonitor, and AttendanceSystem

Entity Objects:

Data-layer components responsible for storing and managing financial and administrative data, including Transaction, Invoice, StaffRecord, and AttendanceRecord

- 1. Strict UML Notation:** All diagrams adhere to standard UML communication diagram conventions.
- 2. Structural Links:** Solid lines represent communication paths between objects, defining how components are connected within the system.
- 3. Message Flow Representation:** Messages indicate interactions between actors, control objects, and entity objects, clearly showing how financial processes are executed

Payment Transaction



- ❖ This diagram shows how a passenger interacts with the airport system to complete a service payment. The system validates the payment details, communicates with an external payment system to process the transaction, and then returns the result to the passenger.
- ❖ It illustrates the structured communication and message flow between all components involved in the payment process

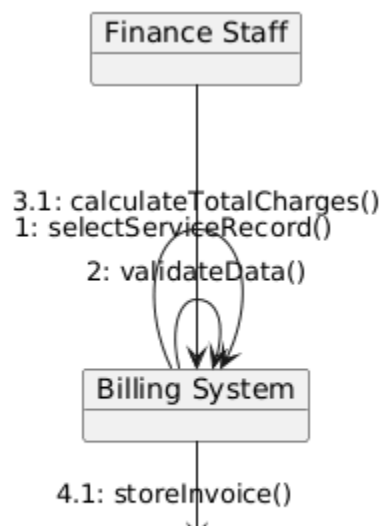
Actors

Passenger – the user selecting a service and providing payment details

System – the main application that handles validation, processing logic, and user interaction

Payment System – the external service responsible for processing the payment and generating transaction records

Airport Billing Process



- ❖ This diagram shows how a passenger requests an airport service, triggering the airport service system to communicate with the invoice system to generate an invoice, which is then returned to the passenger.

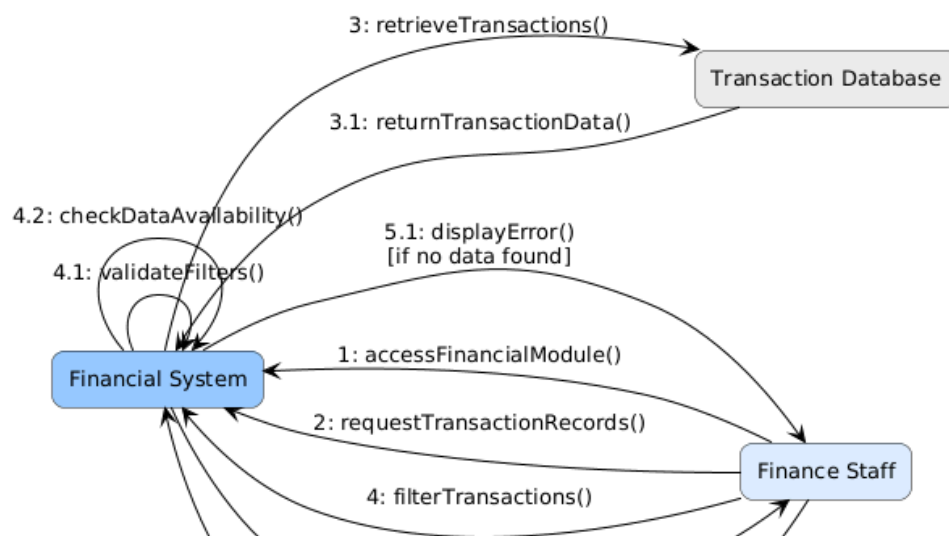
Actors

Finance Staff – the user who initiates invoice generation and reviews billing details

Billing System – the main system that validates data, calculates charges, and manages invoice creation

Database – the storage component that provides usage data and stores generated invoices

Financial Transaction Monitoring



- ❖ This diagram shows how finance staff monitor financial transactions by interacting with the financial system, which retrieves transaction records from the database, validates and filters the data, and then displays the financial results or error messages.

Actors

- **Finance Staff** – the user responsible for reviewing and monitoring financial transactions
-
- **Financial System** – the system that processes requests, filters transactions, validates data, and displays results
-
- **Transaction Database** – the storage system that contains transaction records and financial data

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Betül Kaan

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the Passenger Service Management through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the temporal order of events, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between system objects to ensure a robust and scalable management framework.

Technical Scope

This document focuses on the interaction network between:

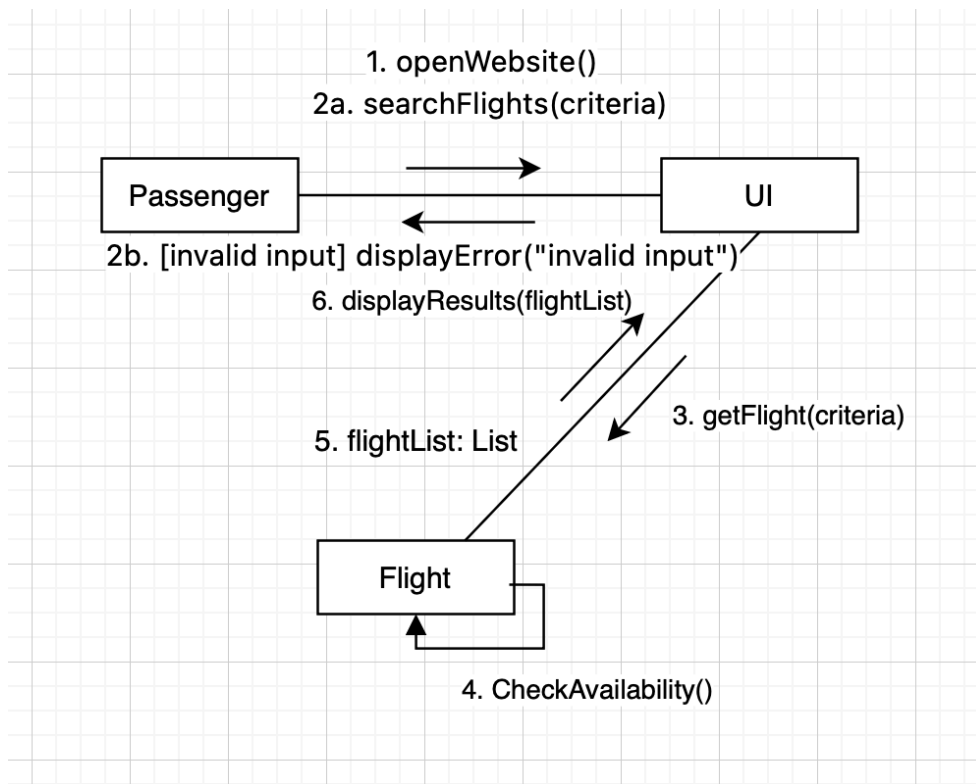
1. *Actor: Passenger*
2. *Boundary Objects: User interfaces used for data input and display — FlightSearchUI, DocumentUploadUI, CheckInUI, BoardingPassUI, GateInfoUI, BaggageTrackingUI, and NotificationSettingsUI.*
3. *Control Objects: Controllers that orchestrate business logic and validation for passenger service operations — UI(present in all UCs), CheckIn (manages window validation and boarding pass generation in UC-03), and FlightNotification(manages preference registration and alert triggering in UC-07).*
4. *Entity Objects: The database layer including Flight, Booking, TravelDocument, BoardingPass, BaggageRecord, BaggageTrackingController, Gate, and FlightNotification.*

System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this document adheres to the following architectural rules:

5. *Strict UML Notation: All models utilize standardized UML collaboration/communication symbols (Object Boxes, Link Lines, and Message Arrows).*
6. *Structural Links: Solid lines represent the static structural pathways over which objects are permitted to communicate.*
7. *Hierarchical Numbering: Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.*
8. *Controller-Centric Routing: Centralized Control objects are used to decouple the User Interface (Boundary) from the Database layer (Entities), ensuring high cohesion and low coupling across the management system.*

UC-01: View Flight Schedules

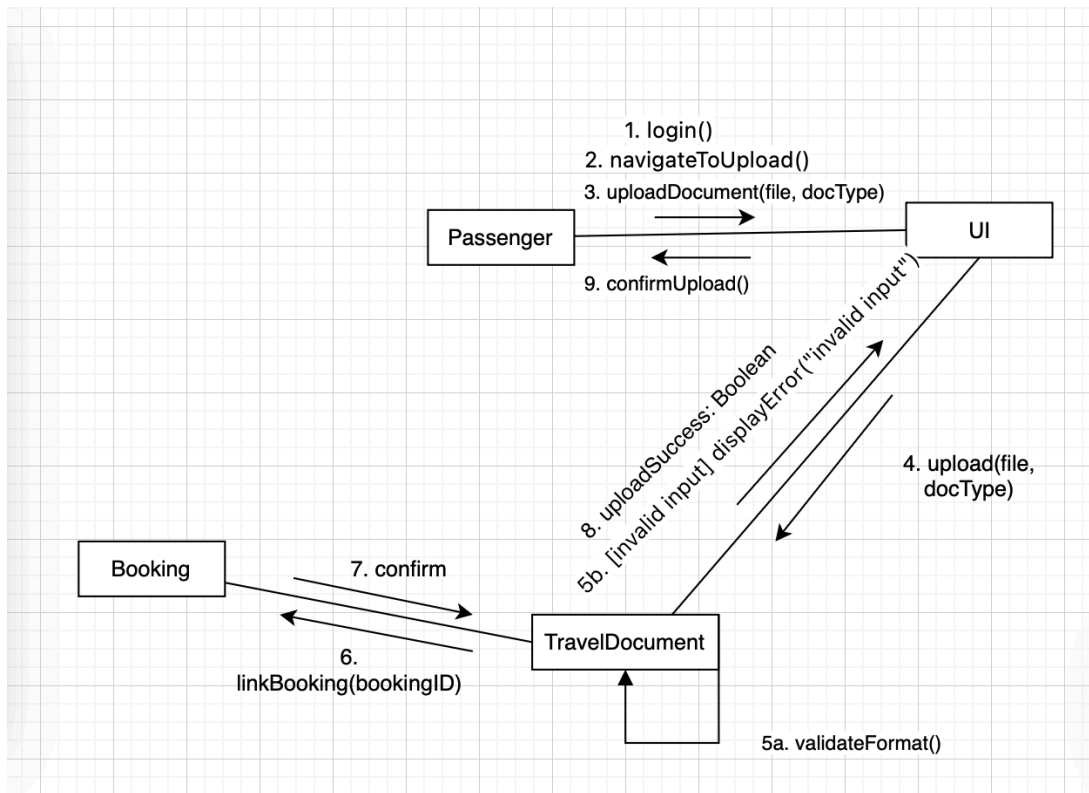


Focus: Flight search and availability retrieval for passengers.

Description: Illustrates the flow of a flight schedule query from the passenger's initial website interaction through the UI layer to the Flight entity, culminating in a filtered list being rendered back to the user.

Key Logic: The UI acts as the central mediator — it receives the search criteria from the Passenger, delegates the lookup via `getFlight(criteria)` (3) to the Flight entity, which internally runs `CheckAvailability()` (4) before returning the `flightList` (5), ensuring only valid and available flights are surfaced in `displayResults()` (6).

UC-02: Upload Travel Documents

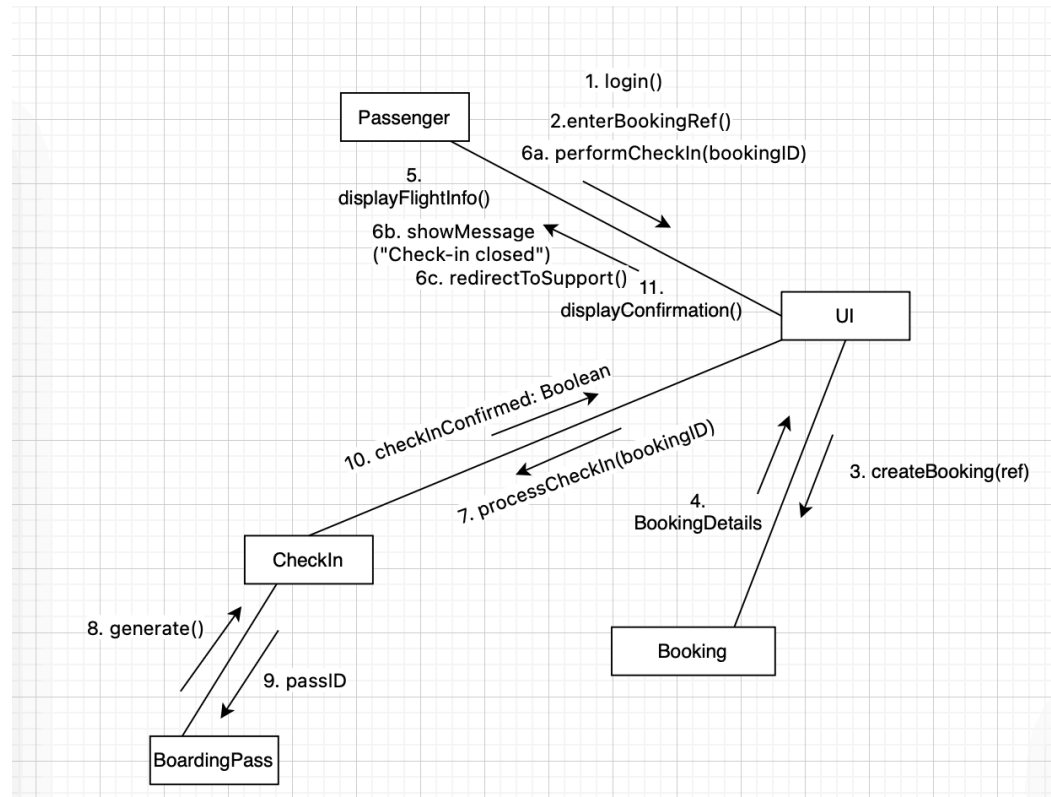


Focus: Secure document submission and booking association for authenticated passengers.

Description: Illustrates the lifecycle of a travel document upload from the passenger's login and navigation through the UI layer to the TravelDocument entity, including format validation, booking linkage via the Booking entity, and a final upload confirmation returned to the passenger.

Key Logic: The TravelDocument entity acts as the processing hub — after receiving the file via `upload(file, docType)` (4), it runs `validateFormat()` (5) before allowing `linkBooking(bookingID)` (6), ensuring documents are only associated with a valid booking once they pass format checks, with the outcome surfaced back to the UI as `uploadSuccess: Boolean` (8).

UC-03: Perform Online Check-In

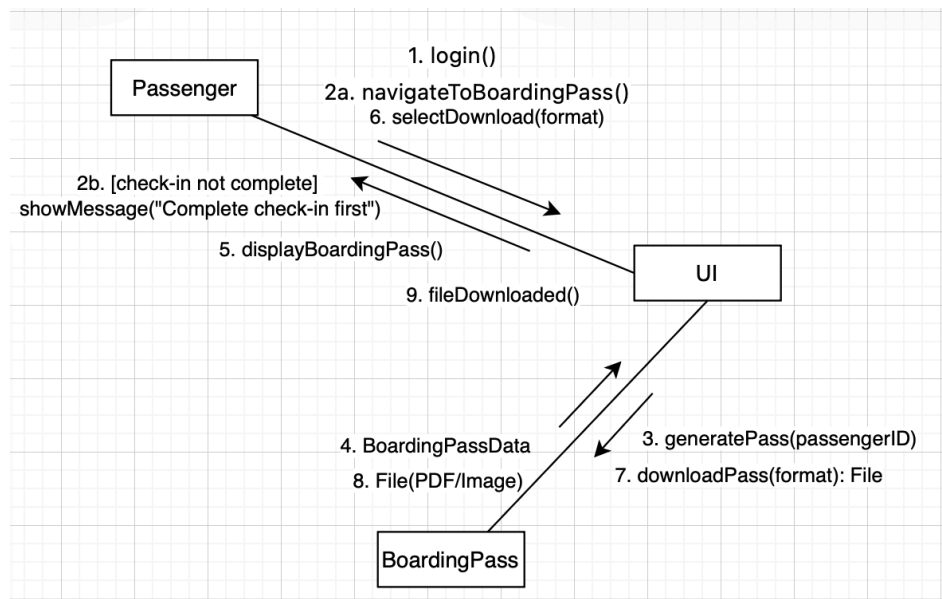


Focus: End-to-end online check-in flow from booking retrieval to boarding pass generation.

Description: Illustrates the multi-step process of a passenger checking in online, beginning with login and booking reference entry through the UI, progressing through booking retrieval, check-in processing, and boarding pass generation, and concluding with a confirmation displayed back to the passenger.

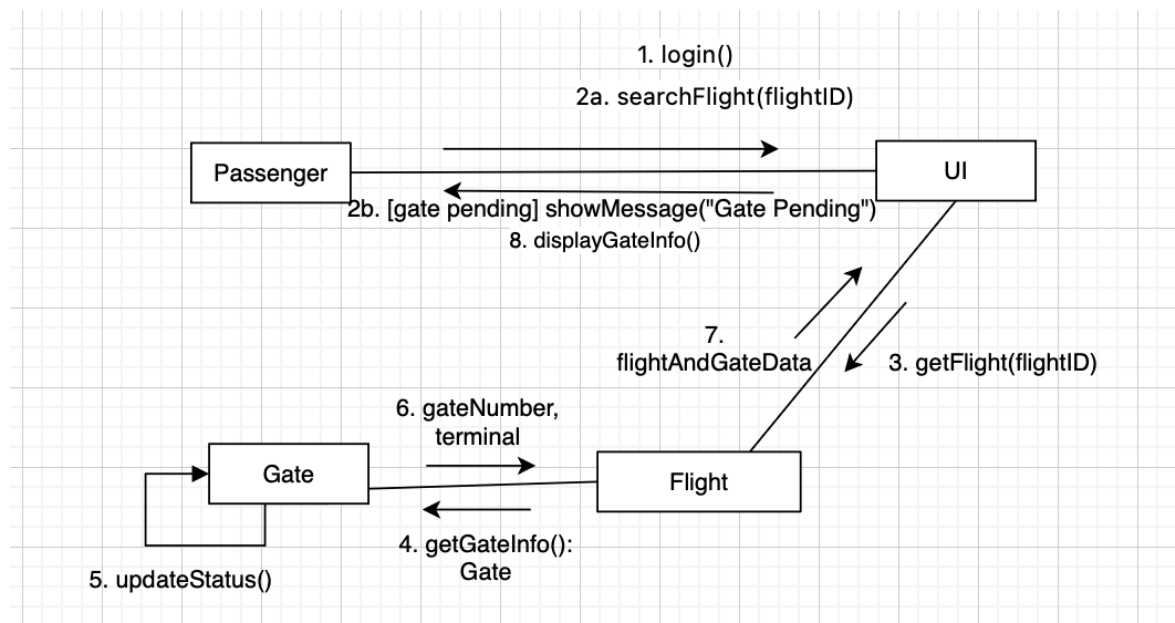
Key Logic: The CheckIn entity serves as the operational core — once the UI triggers processCheckIn(bookingID) (7), the CheckIn entity calls generate() (8) on the BoardingPass entity, which returns a passID (9), enabling the checkInConfirmed: Boolean (10) response to propagate back to the UI. The Booking entity acts as a prerequisite gate, as BookingDetails (4) must be successfully retrieved via createBooking(ref) (3) before the check-in sequence can proceed.

UC-04: Download Boarding Pass



- **Focus:** Boarding pass retrieval and format-specific file delivery to the passenger.
- **Description:** Illustrates the flow of a boarding pass download request from the passenger's login and navigation through the UI layer to the BoardingPass entity, covering pass generation, format selection, and final file delivery back to the passenger.
- **Key Logic:** The BoardingPass entity handles both generation and delivery — the UI first triggers `generatePass(passengerID)` (3), which returns `BoardingPassData` (4) to render the preview via `displayBoardingPass()` (5), and then upon the passenger's `selectDownload(format)` (6), the UI calls `downloadPass(format): File` (7) on the same entity, returning a format-specific `File(PDF/Image)` (8), ensuring the passenger always previews before committing to a download format.

UC-05: Check Assigned Gate Information

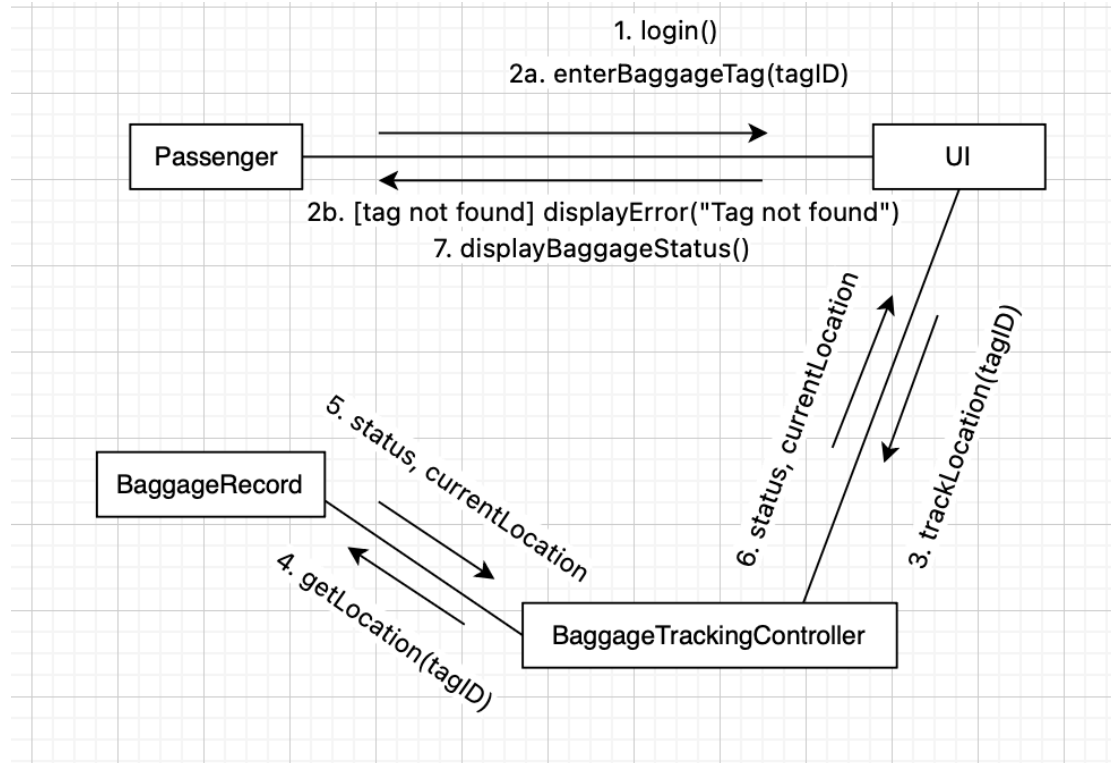


Focus: Real-time gate assignment lookup for a passenger's specific flight.

Description: Illustrates the flow of a gate information query from the passenger's login and flight search through the UI layer to the Flight entity, which delegates to the Gate entity for current assignment data, with the consolidated result returned to the passenger.

Key Logic: The Flight entity acts as the intermediary bridge — after receiving `getFlight(flightID)` (3) from the UI, it calls `getGateInfo(): Gate` (4) on the Gate entity, which first runs `updateStatus()` (5) on itself to ensure current data before returning `gateNumber, terminal` (6). This guarantees that the `flightAndGateData` (7) surfaced to the UI via `displayGateInfo()` (8) always reflects the most up-to-date gate assignment rather than a potentially stale cached value.

UC-06: Track Baggage Status



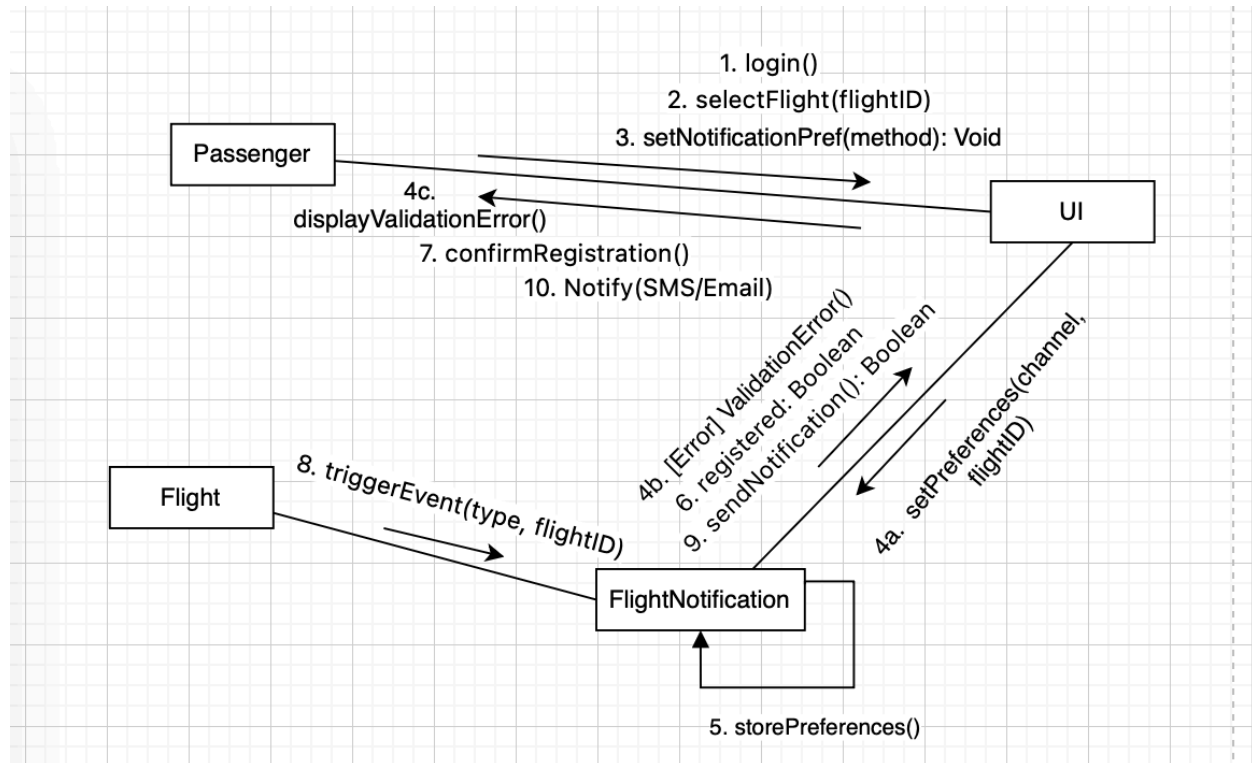
Focus:

Real-time baggage location and status retrieval via tag-based tracking.

Description: Illustrates the flow of a baggage tracking request from the passenger's login and tag entry through the UI to the BaggageTrackingController, which queries the BaggageRecord entity for current location data and propagates the result back to the passenger.

Key Logic: The BaggageTrackingController serves as the dedicated tracking intermediary — upon receiving trackLocation(tagID) (3) from the UI, it calls getLocation(tagID) (4) on the BaggageRecord entity, which returns status, currentLocation (5) sourced directly from the record store. This data is then relayed back to the UI as status, currentLocation (6), ensuring the displayBaggageStatus() (7) presented to the passenger is always fetched live from the BaggageRecord rather than derived from any intermediate cache.

UC-07: Receive Flight Notifications



Focus: Passenger notification preference registration and event-driven alert delivery.

Description: Illustrates the two-phase lifecycle of flight notifications — first, the passenger registers their preferred channel and flight via the UI, and second, the FlightNotification entity autonomously triggers alerts when the Flight entity emits an event, delivering the notification back to the passenger via their chosen method.

Key Logic: The FlightNotification entity drives both phases — during setup, it receives `setPreferences(channel, flightID)` (4) from the UI and internally calls `storePreferences()` (5) before returning `registered: Boolean` (6) to confirm subscription. During the event phase, the Flight entity calls `triggerEvent(type, flightID)` (8) on FlightNotification, which then executes `sendNotification(): Boolean` (9) to dispatch the `Notify(SMS/Email)` (10) to the passenger, ensuring the delivery channel always matches the originally stored preference.

Airport Management System

Technical Design: Collaboration Model Specifications

***Course:** Software Modeling and Design*

***Date:** April 16, 2026*

***Prepared by:** Alvi Elezi*

Document Objective

*The primary objective of this document is to detail the structural organization and message flow of the **Flight Operations Module** through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the passage of time, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between objects.*

Technical Scope

This document focuses on the interaction network between:

- 1. **Actors:** Airport Staff.*
- 2. **Control/Boundary Objects:** System elements responsible for flight operations.*
- 3. **Entity Objects:** The database layer including FlightService, FlightController, and NotificationService.*

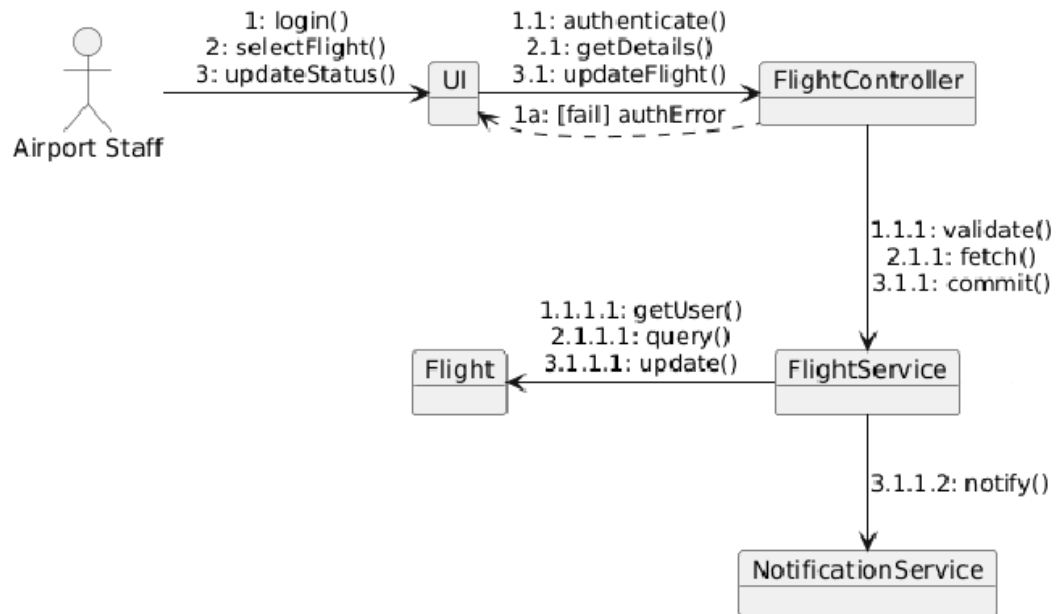
System Interaction Standards

To maintain technical consistency, every collaboration diagram in this document adheres to the following architectural rules:

- 1. **Strict UML Notation:** All models utilize standardized UML collaboration/communication symbols.*
- 2. **Structural Links:** Solid lines represent the structural pathways over which objects can communicate.*
- 3. **Hierarchical Numbering:** Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2, 2.1) to clearly indicate nested operations and sequential logic.*

***Controller-Centric Routing:** Central objects are used to route messages between actors and data entities, keeping the architecture decoupled and highly cohesive.*

1. Update flight status

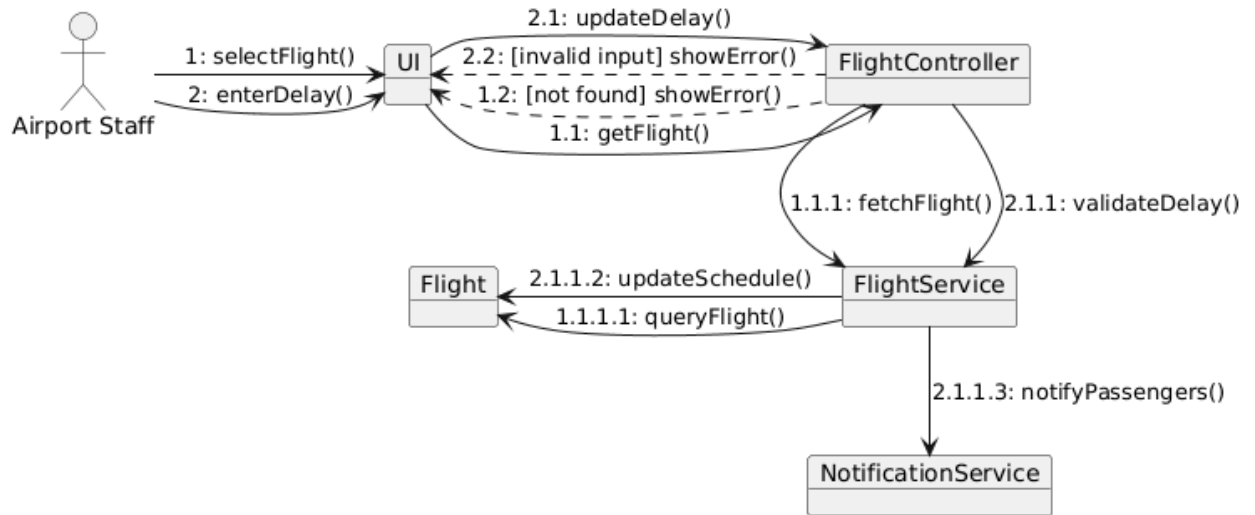


Focus: Flight Status Synchronization

Description: This collaboration diagram illustrates how airport staff initiate and propagate a flight status update across the system. It shows the interaction between the user interface, controller, service layer, and database to ensure the update is consistently applied.

Key Logic: The process is driven by the controller; which validates the request, delegates the update to the service layer, persists the change in the database, and triggers passenger notifications to ensure system-wide consistency.

2. Manage Flight Delays

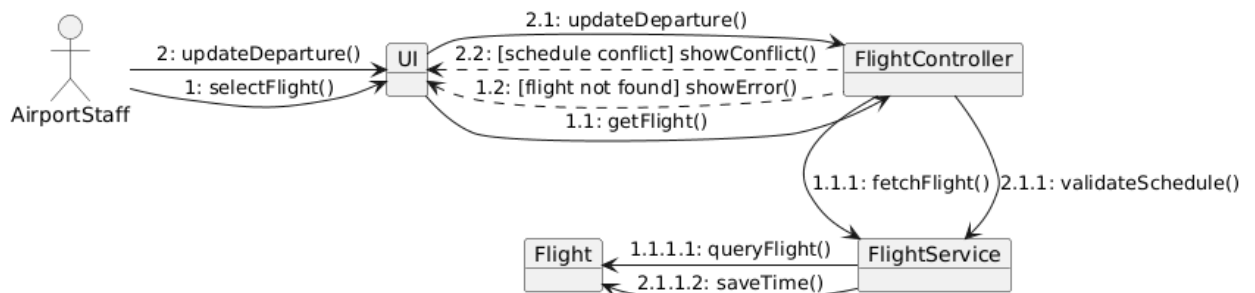


Focus: Delay Processing and Notification

Description: This diagram demonstrates how flight delays are handled within the system, starting from staff input to updating the flight schedule and notifying affected passengers.

Key Logic: The controller coordinates delay validation and delegates schedule updates to the service layer, which ensures that all changes are persisted and communicated through the notification service.

3. Update departure time

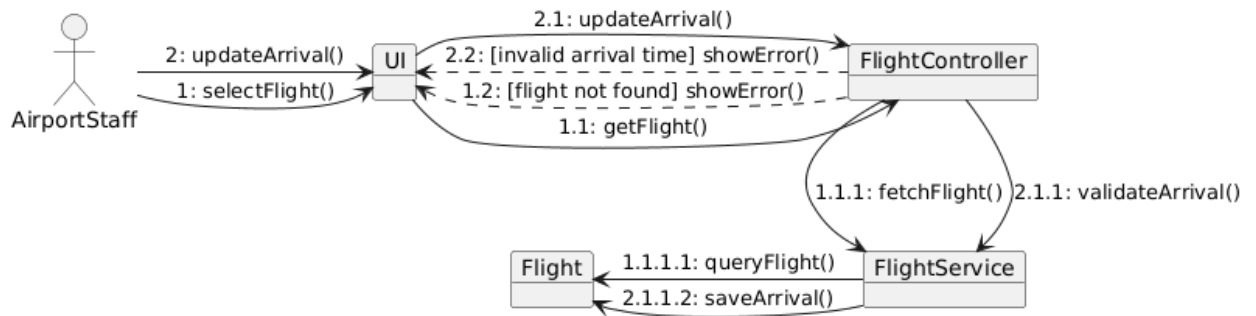


Focus: Departure Time Adjustment

Description: This collaboration diagram represents how modifications to a flight's departure time are processed and stored. It highlights the interaction flow required to retrieve flight data and apply scheduling changes.

Key Logic: The controller enforces scheduling validation rules before the service layer commits the updated departure time to the database, ensuring no operational conflicts occur.

4. Update Arrival Time

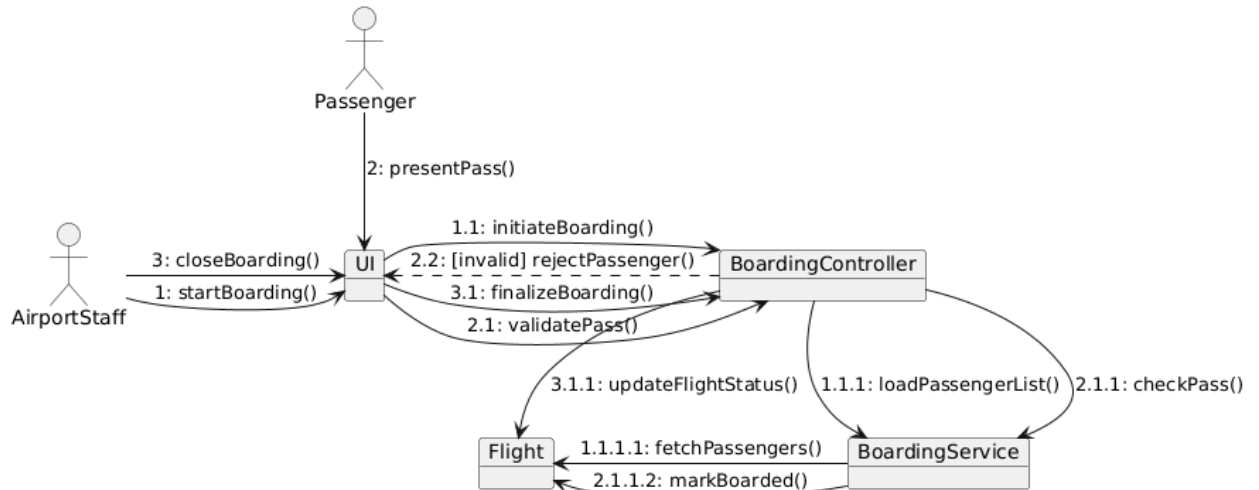


Focus: Arrival Time Management

Description: This diagram shows how updated arrival times are processed within the system, ensuring that flight data remains accurate and synchronized across all components.

Key Logic: The system validates the new arrival time through the controller before updating the flight entity in the database, maintaining consistency with operational constraints.

5. Coordinate Boarding Process

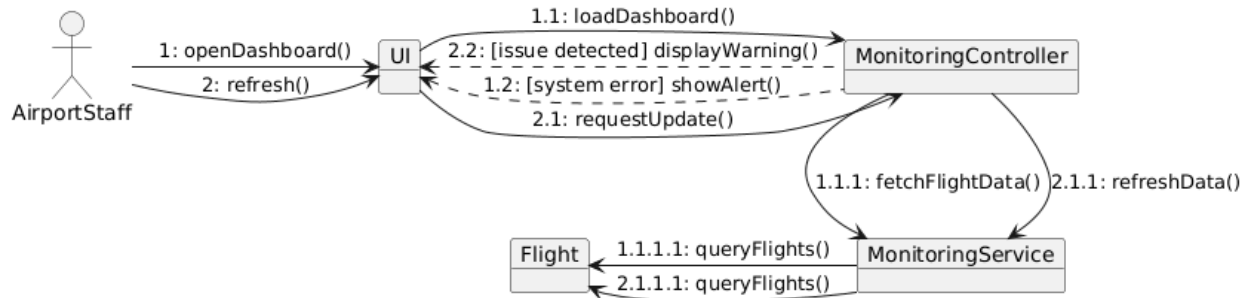


Focus: Boarding Control and Passenger Validation

Description: This collaboration diagram models the boarding process, including passenger validation and status updates as individuals board the aircraft.

Key Logic: The boarding controller manages iterative validation of boarding passes, while the service layer updates passenger records in real time, ensuring accurate tracking of boarding progress.

6. Coordinate Flight Operations



Focus: Real-Time Operational Monitoring

Description: This diagram illustrates how airport staff monitor flight operations through a centralized dashboard, retrieving and refreshing real-time flight data.

Key Logic: The monitoring controller continuously interacts with the service layer to fetch updated flight information from the database, enabling timely detection of operational changes and supporting decision-making.

Airport Management System

Technical Design: Collaboration Model Specifications

***Course:** Software Modeling and Design*

***Date:** April 16, 2026*

***Prepared by:** Ani Velo*

Document Objective

*The primary objective of this document is to detail the structural organization and message flow of the **Security Management Module** through a series of Collaboration (Communication) Diagrams. These diagrams emphasize the architectural relationships and the specific routing of messages between objects to ensure a robust security framework .*

Technical Scope

This document focuses on the interaction network between:

***Actors:** Security Officer, Security Supervisor, and Passenger.*

***Control/Boundary Objects:** SecurityController, AlertManager, and IncidentDashboard.*

***Entity Objects:** PassengerProfile, SecurityLog, and IncidentReport.*

System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this section adheres to the following architectural rules:

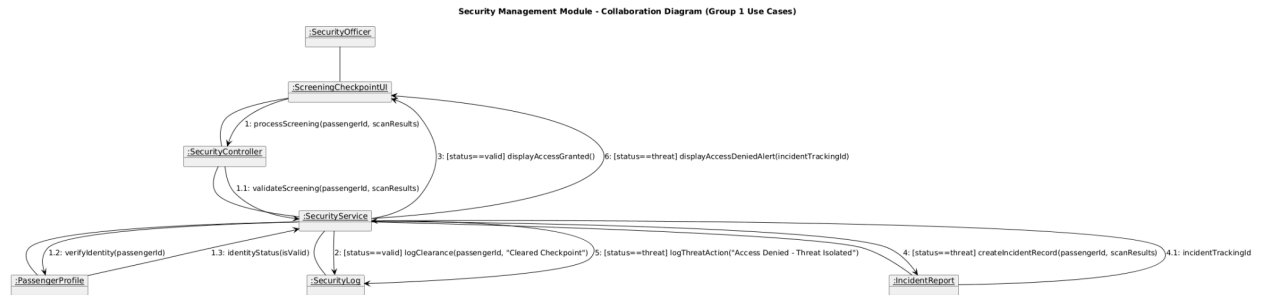
***Strict UML Notation:** All models utilize standardized UML collaboration and communication symbols, including Object Boxes, Link Lines, and Message Arrows.*

***Structural Links:** Solid lines represent the static structural pathways over which objects are permitted to communicate within the security framework.*

***Hierarchical Numbering:** Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.*

***Controller-Centric Routing:** Centralized Control objects (such as the **SecurityController** and **AlertManager**) are used to decouple the User Interface from the Database layer, ensuring high cohesion and low coupling.*

1. UC_SEC_01,02,05 : Process Passenger Security Screening

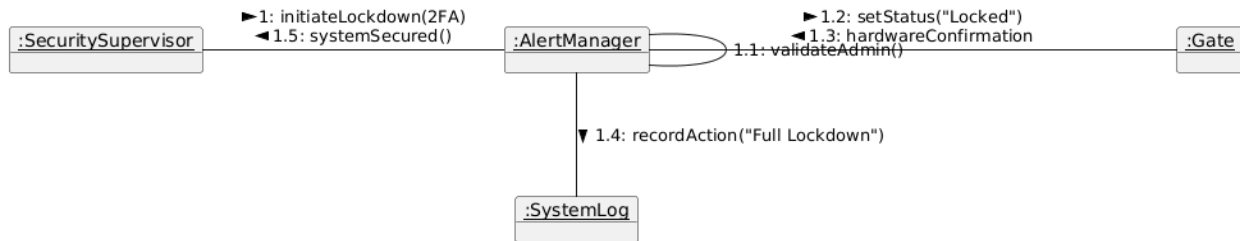


Focus: Passenger Screening and Incident Verification.

Description: This collaboration diagram illustrates how security personnel initiate, verify, and execute a screening workflow across checkpoint boundaries. It shows the unified message routing between the interface UI, security controller, intermediate security service layer, and database entities when a threat or prohibited item is isolated.

Key Logic: The process is driven by the controller, which captures the screening metrics, validates passenger credentials against the profile record, delegates conditional persistence rules to the service layer, and routes automated logs or incident triggers to ensure system-wide security synchronization.

2. UC_SEC_03,04,06: Emergency Facility Lockdown



Focus: High-Priority Command Propagation

Description: Highlights the structural relationships between the Security Supervisor, the AlertManager, and the hardware Gate entities during a critical threat.

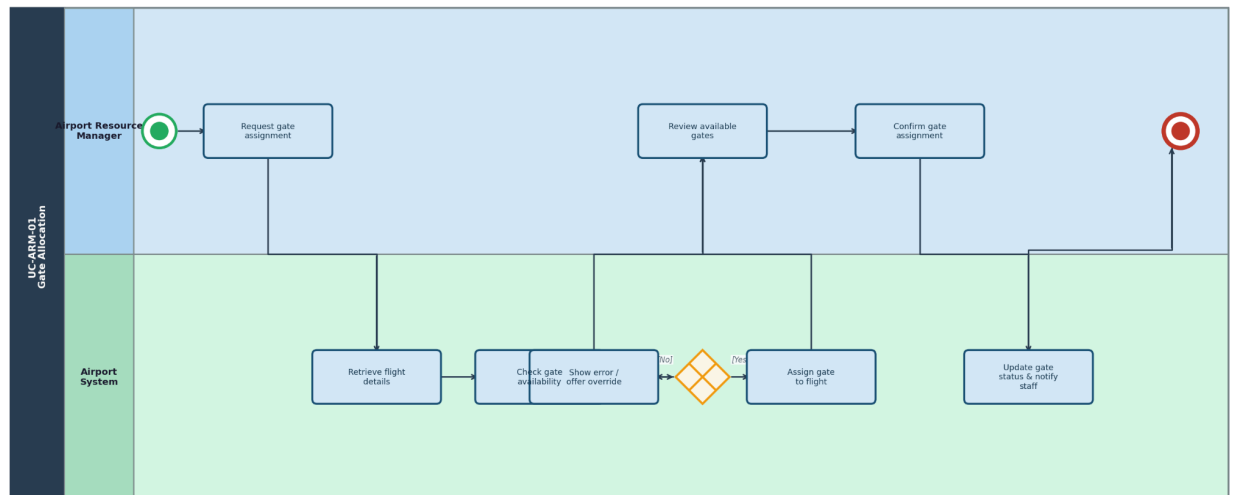
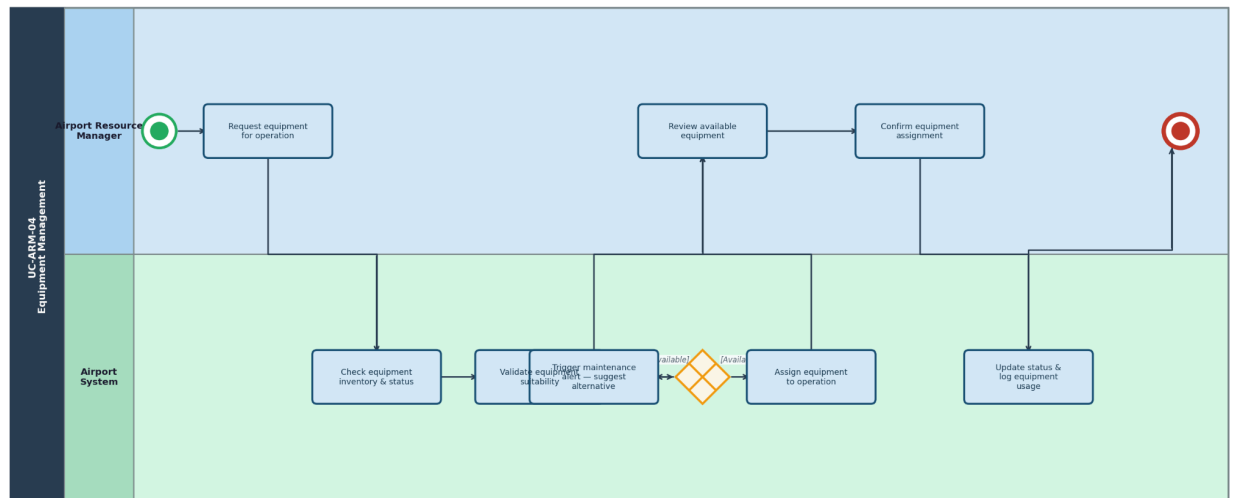
Key Logic:

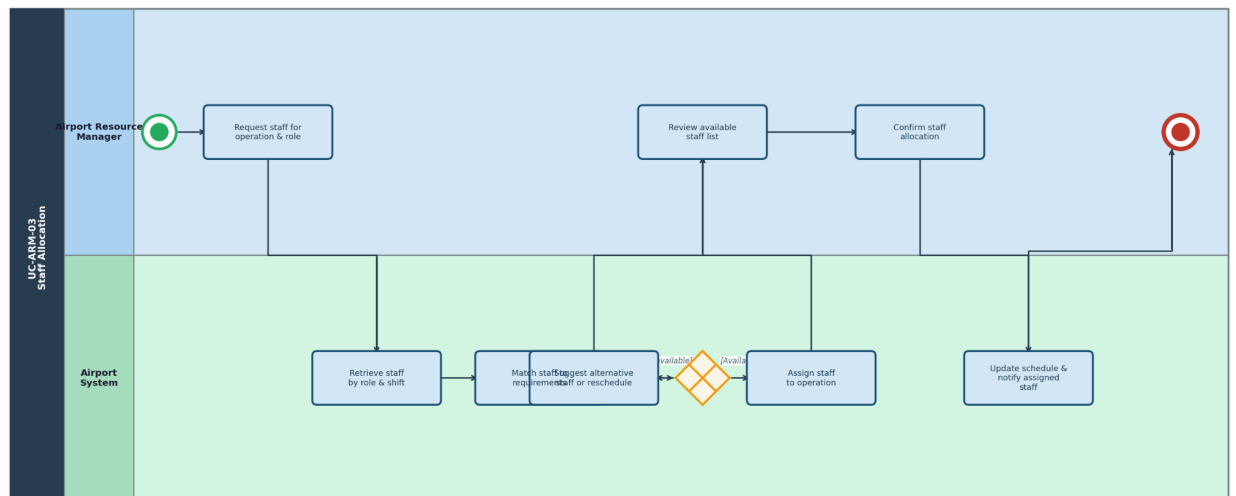
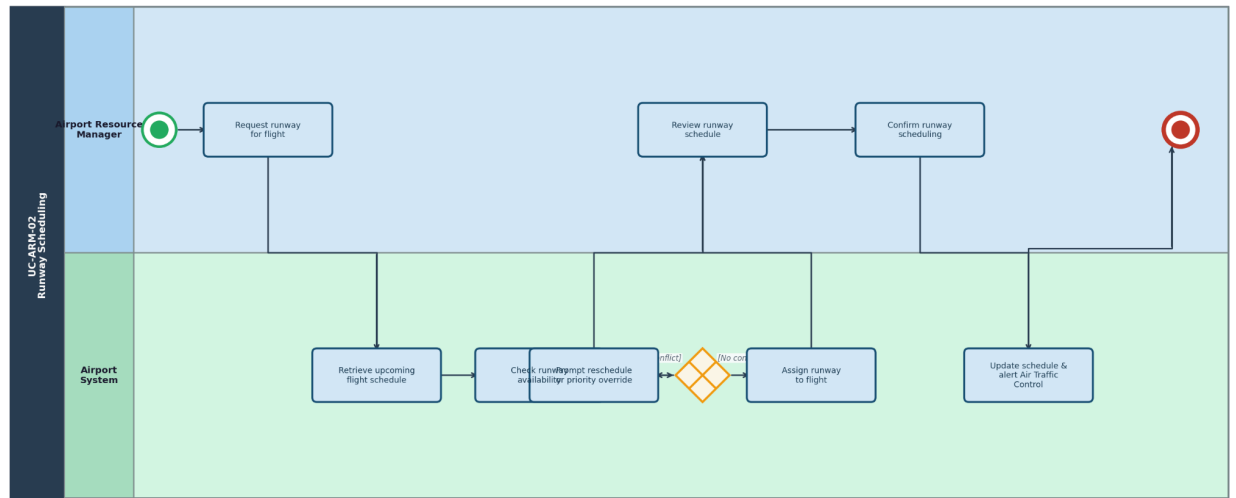
Hierarchical Validation: Nested validation (1.1) ensures that the supervisor's administrative permissions are checked before any hardware command is issued.

Command Propagation: The *AlertManager* handles divergent message routing, transmitting "Lock" signals to physical *Gate* entities while simultaneously triggering global alerts.

Atomic Confirmation: The logic requires a successful *hardwareConfirmation* return from the entity layer before the system state is logged and a success message is rendered.

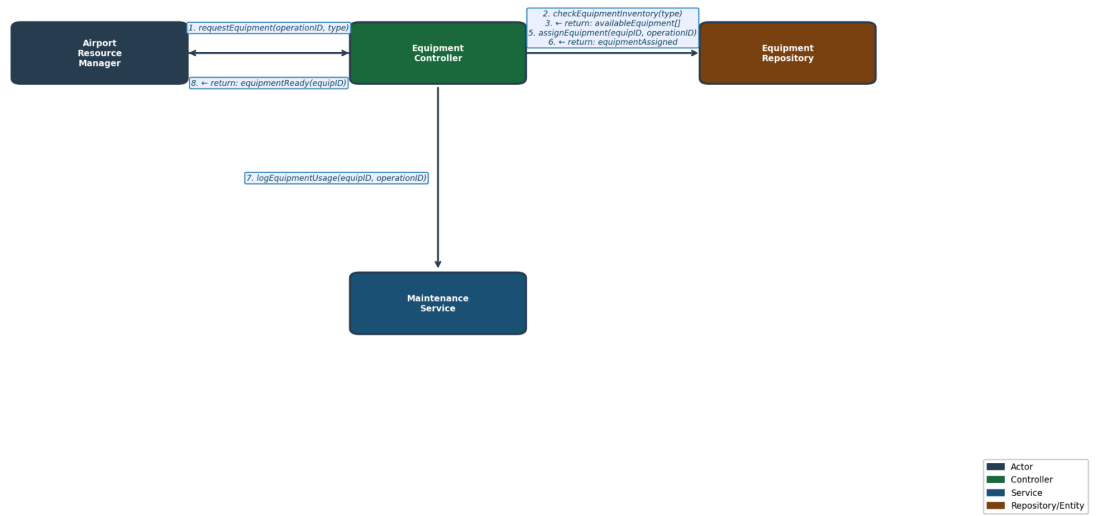
- BPMN Diagram – Airport Resource Management**



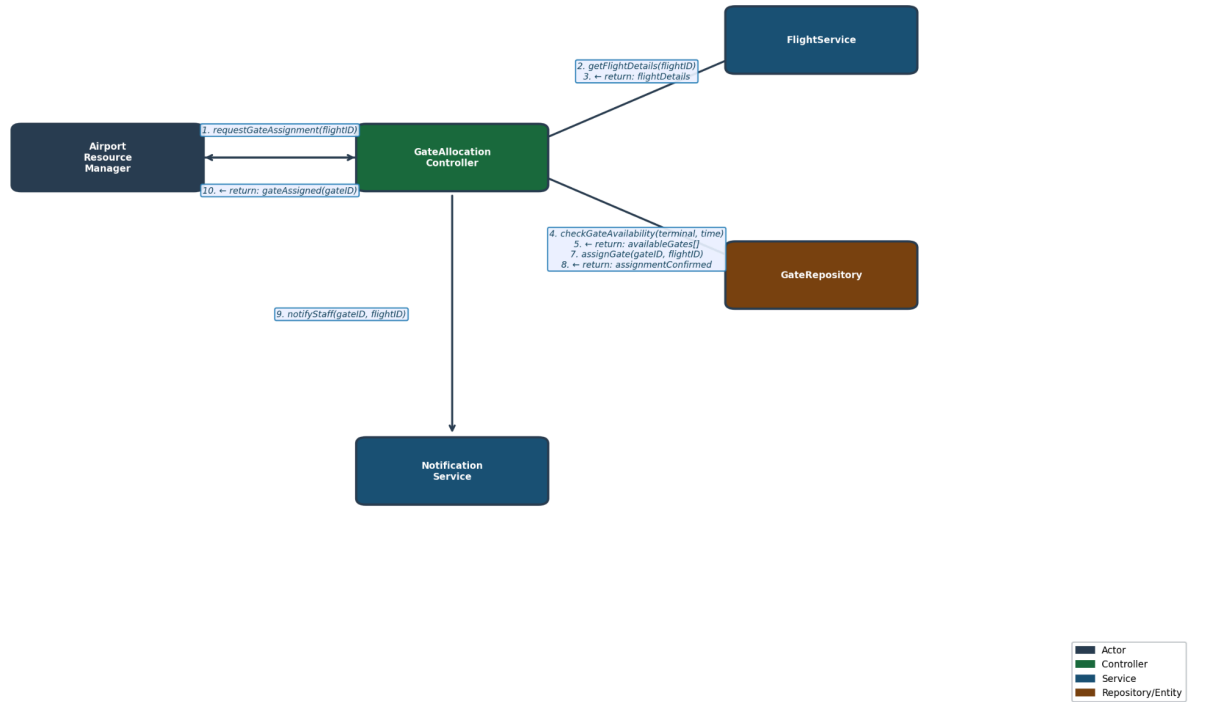


- **Collaboration Diagram – Airport Resource Management**

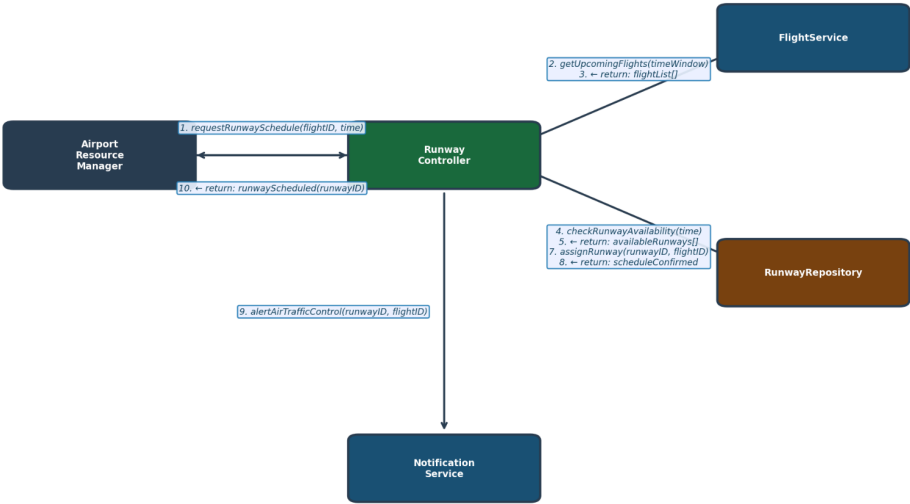
UC-ARM-04: Equipment Management — Collaboration Diagram



UC-ARM-01: Gate Allocation — Collaboration Diagram



UC-ARM-02: Runway Scheduling — Collaboration Diagram



UC-ARM-03: Staff Allocation — Collaboration Diagram

